EchoShield: Detecting AI-Generated Voices in the Modern Soundscape

By

Shahriyar Hasib (011212085)

Md Mehedi Alam Nahi (011212140)

Faisal Ahmed (011212079)

Sahabuddin Shakil (011213039)

Submitted in partial fulfillment of the requirements of the degree of Bachelor of Science in Computer Science and Engineering

November 16, 2025

Department of Computer Science and Engineering
United International University

Abstract

The rise of deepfake audio technologies threatens digital trust. These technologies enable the creation of realistic voice imitations that can lead to fraud, identity theft, and widespread misinformation. Current detection methods show promise when tested on limited datasets; however, they struggle with generalizing across different datasets and fail to make the most of available acoustic information.

We introduce EchoShield, a new dual-branch architecture with 402 million parameters. It combines frozen pre-trained encoders (CLAP, ViT, Wav2Vec2) with trainable forensic modules. Our framework includes five key improvements: (1) an Artifact Detection Module (ADM) with six parallel forensic extractors and confidence estimation, (2) a Dynamic Layer Selection Network (DLSN) that fuses encoder outputs based on attack type, (3) Artifact-Aware Cross-Attention (AACA) that adjusts features based on external confidence, (4) Bidirectional Cross-Branch Interaction (BCBI) that uses mutual information to cut down on redundancy, and (5) Multi-View Contrastive Loss (MVCL) that brings together focal, contrastive, triplet, and consistency goals.

We tested EchoShield on the ASVspoof 2019 LA dataset, which contains 71,237 samples across 19 attack types. EchoShield achieved an accuracy of 95.5%, a 6.8% Equal Error Rate (EER), and a 0.975 AUC-ROC. These results show that our model is better at generalizing and distinguishing between different attack types. Our architecture enhances audio forensics by effectively merging semantic structure analysis with low-level artifact detection. This approach gives a strong framework for confirming the authenticity of digital audio in challenging environments.

Acknowledgments

This work would not have been possible without the input and support of many individuals over the last trimester. We express our heartfelt gratitude to everyone who contributed in any way.

First, we are deeply grateful to Almighty **Allah** for His blessings throughout this journey.

We express our profound thanks to our honorable supervisors, **Dr. Mohammad Nurul Huda**, *Professor and Head of the Department of CSE*, for his invaluable guidance, continuous support, and insightful feedback, and **Md. Tarek Hasan**, *Lecturer, Department of CSE*, for his encouragement and constructive advice that shaped this project significantly.

Our sincere gratitude goes to **Dr. Mohammad Shahriar Rahman**, *Professor, Department of CSE, and Director of CITS*, for his expert suggestions and valuable contributions, which improved the quality and depth of this research.

Last but not least, we owe our family, including our parents, for their unconditional love and immense emotional support.

Publication List

[Optional] The main contributions of this research are either published or accepted or in preparation in journals and conferences as mentioned in the following list:

Journal Articles

1.

Conference Papers

1.

Additional Publications

Following is the list of relevant publications published in the course of the research that are not included in the thesis:

1.

Table of Contents

Table of Contents **vi**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**
1.1 Project Overview . . . 1
1.1.1 Background . . . 1
1.1.2 Problem Statement . . . 2
1.2 Motivation . . . 2
1.3 Objectives . . . 3
1.4 Methodology . . . 3
1.5 Project Outcome . . . 5
1.6 Organization of the Report . . . 6

2 Background **7**
2.1 Preliminaries . . . 7
2.1.1 Self-Supervised Models for Audio Representation . . . 7
2.1.2 Vision Transformers for Spectrogram Analysis . . . 8
2.1.3 Forensic-Inspired Artifact Detection . . . 8
2.1.4 Hybrid Architectures and Advanced Fusion . . . 9
2.2 Literature Review . . . 9
2.2.1 Similar Applications . . . 10
2.2.2 Related Research . . . 11
2.3 Gap Analysis . . . 15
2.4 Summary . . . 16

3 Project Design **17**
3.1 Requirement Analysis . . . 17
3.1.1 Functional and Nonfunctional Requirements . . . 17
3.1.2 Context Diagram . . . 19
3.1.3 Data Flow Diagram Level 1 . . . 20

3.1.4 UI Design . . . 22
3.2 Detailed Methodology and Design . . . 23
3.3 Project Plan . . . 27
3.4 Summary . . . 28

4 Implementation and Results 29
4.1 Experimental Setup . . . 29
4.1.1 Hardware and Software Environment . . . 29
4.1.2 Dataset and Protocol . . . 30
4.1.3 Training Configuration . . . 30
4.2 Results and Performance Analysis . . . 31
4.2.1 Training and Validation Progression . . . 31
4.2.2 Performance on the Evaluation Set . . . 31
4.2.3 Internal Mechanism . . . 32
4.2.4 Comparative Analysis . . . 33
4.2.5 Resource Utilization . . . 33
4.3 Summary . . . 34

5 Standards and Design Constraints 35
5.1 Compliance with the Standards . . . 35
5.1.1 Software Standards . . . 35
5.1.2 Hardware Standards . . . 36
5.1.3 Communication Standards . . . 37
5.2 Design Constraints . . . 38
5.2.1 Economic Constraint . . . 38
5.2.2 Environmental Constraint . . . 39
5.2.3 Ethical Constraint . . . 39
5.2.4 Health and Safety Constraint . . . 40
5.2.5 Social Constraint . . . 40
5.2.6 Political Constraint . . . 40
5.2.7 Sustainability . . . 41
5.3 Cost Analysis . . . 42
5.4 Complex Engineering Problem . . . 45
5.4.1 Complex Problem Solving . . . 45
5.4.2 Engineering Activities . . . 46
5.5 Summary . . . 48

6 Conclusion 49
6.1 Summary . . . 49
6.1.1 Complete Data Flow Summary . . . 50
6.1.2 Performance Summary . . . 50
6.1.3 Architectural Innovations . . . 51

6.1.4 Comparative Performance 51
6.2 Limitations 52
6.2.1 Error Pattern Analysis 52
6.3 Future Work 53
6.3.1 Detailed Future Directions 53

References **57**

List of Figures

1.1 Key Objectives for Deepfake Audio Detection Framework 3
1.2 Basic Methodology 4
1.3 Project Outcome Mapping 5

2.1 Gap Analysis for *EchoShield* 15

3.1 Context Diagram 20
3.2 Data Flow Diagram Level 1 21
3.3 *EchoShield* Interface 22
3.4 Detailed Methodology Architectural Design 23
3.5 High-Level Timeline for *EchoShield* 28

4.1 Training and Validation Progress 31
4.2 Artifact Confidence Distributions and Boxplots 32
4.3 Dynamic Layer Selection Network Weight Distribution 33

5.1 Budget Breakdown Chart 42

List of Tables

2.1 Summary of Similar Applications 10
2.2 Research Papers Review 15

3.1 Functional and Non-Functional Requirements 17
3.2 ASVspoof 2019 LA Dataset Splits 24
3.3 Composition of the Final Augmented Training Set 24

4.1 *EchoShield* Training Hyperparameters 30
4.2 Training and Validation Metrics for 20 Epochs (Selected Epochs) 31
4.3 Comprehensive Performance on the ASVspoof 2019 Evaluation Set (20 Epochs) 32
4.4 Confusion Matrix on the Evaluation Set (20 Epochs) 32
4.5 Performance Comparison with Baselines (20 Epochs) 33
4.6 GPU Memory Usage Breakdown 34
4.7 Overall Performance Comparison 34

5.1 Mapping with complex problem solving. 45
5.2 Mapping with complex engineering activities. 47

6.1 Key Achievements and Contributions of Echoshield 49
6.2 Tensor Shape Transformations 50
6.3 Echoshield Performance Metrics (20 Epochs) 51
6.4 Core Architectural Innovations 51
6.5 Comparative Performance Analysis 51
6.6 Current Limitations and Constraints 52
6.7 Error Pattern Analysis 52
6.8 Future Work Roadmap 53

Chapter 1

Introduction

In recent times, deepfake audio technology has become increasingly powerful, enabling the creation of artificial voices that closely resemble human speech. These developments—fueled by deep learning and artificial intelligence—have found legitimate applications in entertainment, virtual assistants, and content creation. This deepfake audio can be used for fraud, impersonation, and the spread of false information; its misuse can create serious risks to privacy, security, and trust.

Deepfake audio detection is a great struggle, since simultaneous synthesis methods take advantage of the elasticity of human speech to get beyond conventional detection systems. Early detection attempts frequently lacked the sophistication of more advanced deepfake techniques, depending instead on simple feature extraction and traditional machine learning models.

The goal of this research is to overcome these constraints by creating a strong framework to identify fake audio using cutting-edge techniques. This study aims to detect the distinct artifacts found in synthetic speech by utilizing hybrid deep-learning models and collecting important spectral and temporal properties from audio data. The goal is to develop a system that can handle a variety of datasets and synthesis methods while remaining accurate and generalizable.

Assuring that the authenticity of audio data can be consistently confirmed in controlled settings, the project's results are anticipated to improve the security of digital audio content and advance the field of audio forensics.

1.1 Project Overview

1.1.1 Background

Artificial Intelligence is widely used in deepfake audio technology to create realistic-sounding synthetic voices that can mimic real people. And these technologies can be used

for pleasure, but when misused, they can also be extremely dangerous. One of the main challenges in detecting these forgeries lies in a growing arms race between synthesis and detection. Early detection methods are often dependent on handcrafted acoustic features such as Mel-Frequency Spectral Coefficients (MFCCs). Although effective in capturing the phonetic content of speech, they are often not responsive to the subtle, almost unseen artifacts that differentiate a state-of-the-art synthetic voice from real speech. Modern generative models are explicitly trained to minimize these very discrepancies, rendering conventional methods increasingly obsolete and creating an urgent need for more adaptive detection frameworks.

The research aims to increase the accuracy of deepfake detection by utilizing ultramodern techniques, such as feature engineering and hybrid deep learning models. The project's focus is academic and research-oriented, prioritizing dataset experimentation and document evaluation over real-time implementation to build a reliable detection system.

1.1.2 Problem Statement

The development of deep learning technologies has led to the production of extremely realistic deepfake audio and poses serious threats to the original audio, safety, and confidentiality of digital communications. Synthetic voices that are nearly indistinguishable from authentic speech have become an increasing threat to uses associated with fraud, impersonation, and disinformation. Despite progress in detection techniques, existing methods struggle to generalize across diverse datasets and fail to effectively address the subtle artifacts embedded in audio generated by advanced synthesis methods. This challenge highlights the urgent need for improved solutions to protect digital audio integrity.

1.2 Motivation

As deepfake audio technology has grown in popularity, it has become increasingly difficult to verify the legitimacy of digital conversations. There is a serious risk to privacy, security, and public trust when sophisticated AI algorithms are able to produce artificial voices that accurately replicate actual human speech. The financial, media, and public safety sectors can all be impacted by the use of these deepfake audios as a weapon for fraud, impersonation, and disinformation.

The opportunity to advance machine learning and signal processing methods in order to detect subtle and distinctive artifacts incorporated into synthetic audio provides the computational incentive for solving this issue. Using cutting-edge algorithms that can process high-dimensional, temporal, and spectral characteristics of voice data is necessary to detect these aberrations.

Resolving this issue has broad advantages. It makes digital verification systems more reliable, protects people and organizations from deepfake-based abuses, and fosters confidence in digital ecosystems. Additionally, the study advances the science of audio forensics and strengthens our defenses against malevolent AI applications while promoting safe and moral technological development.

1.3 Objectives

Our research focuses on finding and extracting distinctive features in artificial speech, creating sophisticated machine-learning models, assessing the robustness of various datasets, improving performance for efficiency and accuracy, and advancing the field of audio forensics by providing scalable solutions to prevent the abuse of deepfake audio (see Figure 1.1).

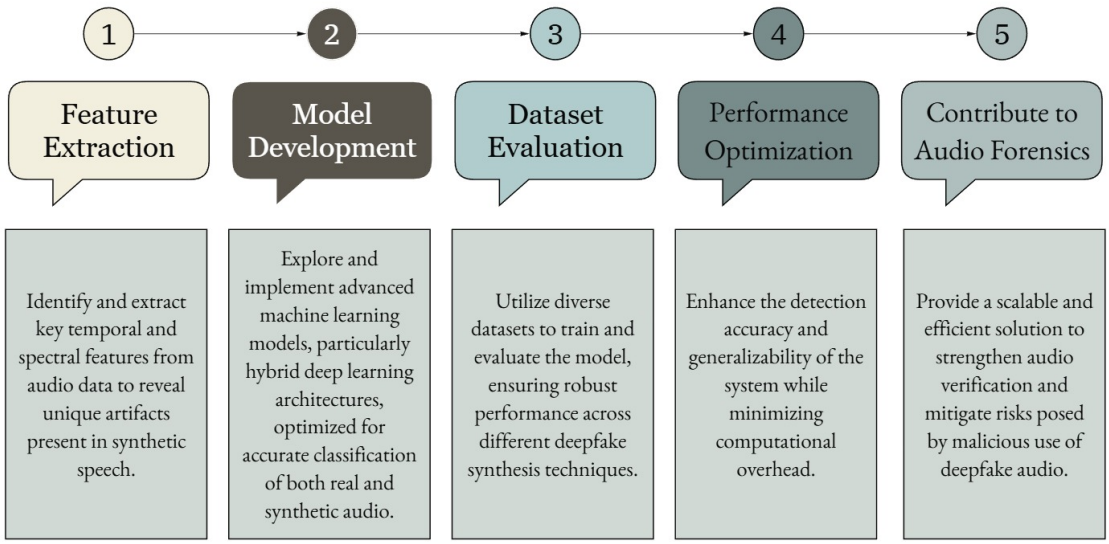

Figure 1.1: Key Objectives for Deepfake Audio Detection Framework

1.4 Methodology

The *EchoShield* methodology is designed to create a robust framework for detecting deepfake audio. The process begins with collecting diverse datasets, including ASVspoof, WaveFake/LJSpeech, and Release-In-The-Wild. Processes like segmentation, noise filtering, normalization, and data augmentation all play a crucial role in ensuring the input data is both high-quality and reliable for analysis. Feature extraction focuses on three areas: spectrogram-based features to analyze time-frequency patterns, auditory filters to mimic human hearing, and temporal features to capture rhythm and energy.

A hybrid model architecture combines CNNs for spatial analysis, RNNs for sequential pat-

tern detection, and transformers to capture long-range dependencies. Pre-trained models are leveraged to extract semantic embeddings for better classification. Ensemble learning integrates predictions from these models to improve robustness.

Performance metrics, such as F1-score, recall, accuracy, and precision, are used to assess how effective the suggested framework is. The methodology ensures strong performance across datasets, balancing accuracy and scalability for advanced deepfake detection.

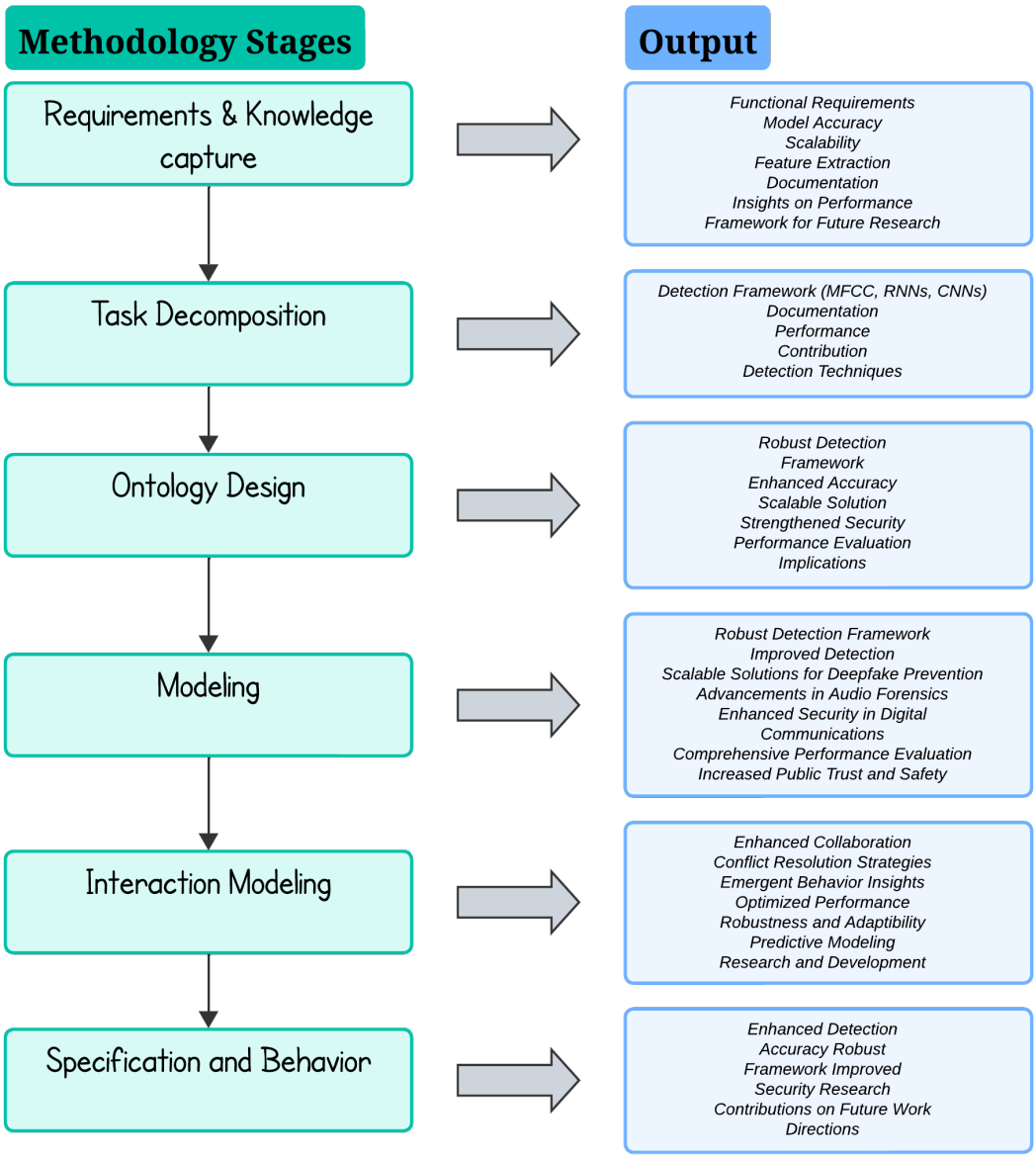

Figure 1.2: Basic Methodology

1.5 Project Outcome

Development of a Robust Deepfake Audio Detection Framework for Enhanced Security, Authenticity, and Forensic Applications. The project outcome is mapped in Figure 1.3.

Figure 1.3: Project Outcome Mapping

1.6 Organization of the Report

This report is structured with six chapters, every chapter addressing critical aspects of the research on detecting deepfake audio. The chapters are organized as follows:

- **Chapter 1: Introduction**
  Here introduces the research topic, *"EchoShield: Detecting AI-Generated Voices in the Modern Soundscape,"* highlighting the growing concern over synthetic audio and its impact on privacy, security, and misinformation. This chapter outlines the research problems, methodology, objectives, and structure of the thesis.

- **Chapter 2: Background**
  This chapter provides a review of the evolution of AI-generated audio and existing detection methods. It identifies limitations in current work approaches and highlights a research gap that motivates this study.

- **Chapter 3: Methodology**
  The design and framework of the proposed detection system are discussed, detailing feature extraction methods, machine learning models, and system architecture. This chapter includes diagrams to illustrate the workflow and outlines the research plan.

- **Chapter 4: Results and Analysis**
  The outcomes of this chapter, which involves implementing the *EchoShield* system, include performance evaluations and comparative analysis. Key findings, system accuracy, and areas for improvement are highlighted.

- **Chapter 5: Discussion and Standards**
  This chapter explores the broader implications of the findings, addressing design constraints, ethical considerations, and compliance with relevant standards. A cost analysis and strategies to address system limitations are also included.

- **Chapter 6: Conclusion and Future Work**
  Finally, it summarizes the research contributions, acknowledges study limitations, and suggests directions for future work, such as real-time detection and advanced model integration.

Chapter 2

Background

This chapter provides a broad overview of the key concepts and technologies that form the foundation of this work. It moves beyond a general discussion of deep learning to focus on the specific, state-of-the-art techniques applied in our dual-branch architecture for deepfake audio detection. We will first set up the preliminaries by dissecting the roles of self-supervised learning models like Wav2Vec2 and CLAP, vision-based transformers for spectrogram analysis, and expert forensic techniques for artifact detection. Subsequently, the chapter will review existing literature, critically analyzing the limitations of prior work and identifying the research gaps that *EchoShield* is designed to address. This basic knowledge is essential for understanding the novel contributions of our methodology, which combines high-level semantic analysis with low-level artifact detection to create a robust and generalizable system.

2.1 Preliminaries

This section provides the essential technical background required to understand the advanced methodologies and hybrid model architectures discussed in this thesis. The focus is on the specific deep learning paradigms and pretrained models that are foundational to *EchoShield*'s dual-branch design. We will explore how large-scale, self-supervised models provide rich feature representations from raw audio and spectrograms and how specialized convolutional techniques can be adapted from digital forensics to identify manipulation artifacts.

2.1.1 Self-Supervised Models for Audio Representation

A cornerstone of modern audio processing is the use of large-scale, pre-trained models that learn rich, general-purpose representations of audio without requiring massive labeled datasets. These models form the backbone of *EchoShield*'s Structure Branch.

i. **Wav2Vec 2.0:** Developed by Facebook AI, Wav2Vec 2.0 [1] is a self-supervised framework that learns representations directly from raw audio waveforms. It oper-

ates by masking parts of the audio signal and training the model to predict the quantized latent representations of the masked regions. This process forces the model to learn high-level contextual information, including phonetic structure, prosody, and speaker characteristics, making its embeddings highly effective for downstream tasks like deepfake detection. In our work, we utilize the frozen embeddings from the `facebook/wav2vec2-base-960h` model to capture the temporal and structural integrity of speech.

ii. **Contrastive Language-Audio Pre-training (CLAP):** CLAP models, such as `laion/clap-htsat-unfused` [2], learn multi-modal embeddings that connect audio with natural language descriptions. By training on multiple datasets of audio clips and their corresponding text captions, CLAP learns to associate sounds with their semantic meaning. Detecting deepfakes, CLAP provides a powerful semantic prior. A synthetic voice might sound human-like but may lack the subtle acoustic cues associated with the described context (e.g., the sound of a person speaking in a large room). This provides a unique, high-level signal for detecting inconsistencies.

2.1.2 Vision Transformers for Spectrogram Analysis

Treating audio spectrograms as images allows us to use powerful architectures from computer vision. This is a key component of both the Structure and Artifact branches of our model.

i. **Vision Transformer (ViT):** The Vision Transformer [3] adapts the Transformer architecture, which is basically designed for natural language processing, to image analysis. It works by dividing an image, or in our case, a spectrogram image, into a sequence of fixed-size patches, which are then linearly embedded and fed to the Transformer encoder. The self-attention mechanism allows the model to learn global relationships between different time-frequency patches, making it exceptionally good at capturing long-range dependencies and subtle textural patterns that are often indicative of synthetic audio. We employ a pre-trained `google/vit-base-patch16-224` model to analyze the visual structure of spectrograms.

2.1.3 Forensic-Inspired Artifact Detection

While self-supervised models excel at capturing high-level structure, they may overlook the low-level, almost imperceptible traces left by the synthesis process. *EchoShield*'s Artifact Branch is explicitly designed to hunt for these forensic clues using specialized techniques.

i. **Constrained Convolution (BayarConv2d):** Adapted from image forgery detection, the Bayar convolution [4] is a constrained convolutional layer designed to suppress an image's content and amplify manipulation artifacts. By forcing the central weight of the kernel to be -1 and the sum of all other weights to be 1, it acts as a

high-pass filter that learns to predict a pixel from its neighbors. In deepfake audio, this helps to reveal inconsistencies in the spectrogram's noise floor and texture that are invisible to standard CNNs.

ii. **Spatial Rich Model (SRM) Filters:** SRM [5] provides a set of handcrafted, non-learnable high-pass filters designed to capture subtle noise patterns used for analysis. These filters are highly effective at extracting remaining noise patterns, compression artifacts, and other minute traces that are qualities of generative processes. By incorporating these filters, our model is primed to detect forensic evidence without needing to learn these complex relationships from scratch.

2.1.4 Hybrid Architectures and Advanced Fusion

The novelty of *EchoShield* lies not just in its components but in how they are integrated.

i. **Dual-Branch Design:** Our architecture divides the detection job into two complementary sub-problems: the Structure Branch evaluates the semantic and temporal coherence of the speech, and the Artifact Branch seeks out low-level forensic traces.

ii. **Dynamic and Interactive Fusion:** Instead of easy concatenation, we use innovative fusion methods. The Dynamic Layer Selection Network (DLSN) adaptively weights the outputs of many pre-trained encoders based on the predicted type of assault. In addition, Artifact-Aware Cross-Attention (AACA) and Bidirectional Cross-Branch Interaction (BCBI) allow the two branches to mutually advise. This allows the model to, for example, focus its search for artifacts in areas where the semantic structure appears to be weak.

iii. **Multi-Objective Loss Function:** We train this sophisticated system using a Multi-View Contrastive Loss (MVCL). This aim combines a primary classification loss (Focal Loss) with a number of auxiliary losses, including contrastive (NT-Xent), triplet, and consistency losses. This multi-task learning technique ensures that the model not only learns to accurately categorize but also creates a rich, well-structured embedding space that distinguishes various sorts of fakes from one another and from actual audio.

2.2 Literature Review

The fast emergence of deepfake audio technology has created substantial issues in maintaining the authenticity and security of voice-based systems. Existing detection approaches sometimes fall short in terms of accuracy, scalability, and resilience, especially as synthetic audio gets more complex, emulating both temporal coherence and spectral subtleties.
Recent research has addressed these challenges through a range of approaches. Techniques like Temporal Deepfake Location (TDL) [10] utilize Wav2Vec2 embeddings and temporal

convolution to detect partially spoofed audio at the frame level. Similarly, MOS-FAD [8] incorporates perceptual artifacts using Mean Opinion Scores (MOS) and novel fusion strategies such as Gated MLP for refined classification.

Spectrogram-based models [6][11][12][14][15][21][22] employ STFT, LFCC, Wavelet transforms, and hybrid architectures like CNNs, RNNs, and C-RNNs to identify spectral anomalies. The Codecfake dataset [7] focuses on codec-induced artifacts, utilizing LCNN and SSL-based techniques for generalized detection. Papers like [26] and [28] further explore how self-supervised models (e.g., Wav2Vec2, XLS-R, and Whisper) enhance performance through richer temporal embeddings.

Several studies [20][23][27] explore Mixture of Experts, transfer learning, and temporal convolution networks, demonstrating the utility of combining frozen Wav2Vec2 models with ensemble strategies to improve generalization across datasets.

Despite these advancements, current methods often analyze only isolated aspects—temporal [10][26], spectral [6][11][12], or perceptual [8]—without combining them into a unified system. Additionally, robustness to real-world variability, background noise, and unseen synthesis methods remains a gap [16][19][29]. These limitations highlight the need for integrated approaches leveraging adaptive feature extraction, multi-modal fusion, and lightweight transformer-based architectures to ensure scalable and reliable detection of audio deepfakes.

2.2.1 Similar Applications

Application	**Description**
Resemble Detect [6]	Research uses deep neural networks to examine time-frequency embeddings and detects small anomalies in AI-generated audio with above 98% accuracy. Provides strong defenses against manufactured misinformation.
PlayHT Voice Classifier [7]	Uses machine learning algorithms to distinguish between real and synthetic voices. Offers detailed analysis reports with a user-friendly interface, suitable for both professional and casual use.
Deepware Scanner [8]	An open-source tool combining machine learning and spectral analysis for detecting synthetic audio and audiovisual deepfakes. Supports applications like social media verification and forensic analysis.
Loccus AI [9]	Focuses on fraud prevention by integrating AI voice detection into communication systems. Supports multiple formats and languages.
Reality Defender [10]	Detects audio deepfakes by analyzing synthesis artifacts. Offers solutions for governments, enterprises, and platforms to identify manipulated audio.

Table 2.1: Summary of Similar Applications

2.2.2 Related Research

Title	**Methodology**	**Result**
Deepfake Audio Detection Using Spectrogram-Based Features and Ensemble of Deep Learning Models [11]	∗ Spectrogram-Based Feature Extraction: STFT, LFCC, Wavelet. ∗ End-to-End Models: CNN Baseline, RNN Baseline (Bi-LSTM), C-RNN Baseline. ∗ Transfer Learning: ResNet, ConvNeXt, Swin-Tiny. ∗ Audio Embedding Models: Whisper, SpeechBrain, Pyannote.	EER: 0.03%
The Codecfake Dataset and Countermeasures for the Universal Detection of Deepfake Audio [12]	∗ Feature Extraction: Mel-Spectrograms, Wav2Vec2-XLS-R Embeddings. ∗ Classification: CNN, LCNN. ∗ AASIST: Temporal modeling techniques (LSTM) + softmax activation.	EER: 0.616%
MOS-FAD: Improving Fake Audio Detection via Automatic Mean Opinion Score Prediction [13]	∗ Train an MOS predictor. ∗ Feature extraction and detection: Wav2Vec. ∗ Fusion Techniques: Score fusion, embedding fusion. ∗ Gated MLP.	EER: 0.96%
SPECDIFF-GAN: A Spectrally-Shaped Noise Diffusion GAN for Speech and Music Synthesis [14]	∗ Noise Diffusion Preprocessing. ∗ Generator: HiFi-GAN Architecture. ∗ **Discriminators:** Multi-Period and Multi-Resolution Discriminators.	Not mentioned
An Efficient Temporary Deepfake Location Approach Based on Embeddings for Partially Spoofed Audio Detection [15]	∗ Feature Extraction: Wav2Vec2. ∗ Embedding Similarity Module. ∗ Temporal Convolution Operation. ∗ Classification: 1-D CNN.	Not mentioned

A Comprehensive Survey with Critical Analysis for Deepfake Speech Detection [16]	∗ Spectrogram-Based: STFT, Mel-Spectrogram. ∗ Auditory Filters: LFCC, Wavelet. ∗ End-to-End Models: CNN, RNN, C-RNN. ∗ Ensemble Models: Multiple Inputs, Multiple Branches. ∗ Transfer Learning: ResNet, MobileNet, EfficientNet. ∗ Audio Embedding Models: XLS-R, Hubert, WavLM, Whisper.	EER: 0.03%
Is Synthetic Voice Detection Research Going into the Right Direction [17]	∗ STFT. LFCC, Wavelet. CNN Baseline, RNN Baseline (Bi-LSTM), and C-RNN Baseline. ResNet, ConvNeXt, and Swin-Tiny. Whisper, SpeechBrain, Pyannote.	EER: 0.05%
AntiDeepFake: AI for Deep Fake Speech Recognition [18]	∗ Mel spectrogram. Pitch, Shimmer, RMSS, Jitter, Chroma STFT, Spectral Centroids, Spectral Bandwidths, Rolloffs, Zero Crossing Rates, MFCC (Mean, Standard Deviation). ∗ Formants (F1, F2, F3, F4).	Not mentioned
Detecting Deep Fake Voice Using Machine Learning [19]	∗ Spectrogram-based features: STFT, MFCC, and wavelet. ∗ Temporal and frequency domain analysis. ∗ CNN, Bi-LSTM, Transfer Learning, Ensemble Learning. ∗ Pre-trained architectures: Whisper, SpeechBrain.	EER: 0.02%
Detecting Deepfake Voice Using Explainable Deep Learning Techniques [20]	∗ STFT, Mel-filterbanks (applied for perceptual emphasis on low-frequency spectra). ∗ CNN, LSTM, XAI Methods, Deep Taylor Decomposition. Integrated Gradients. Layer-Wise Relevance Propagation (LRP). Griffin-Lim Algorithm.	EER: 0.03%

Audio Deep Fake Detection with Sonic Sleuth Model [21]	∗ Audio preprocessing to handle noise. ∗ CNN for feature extraction and classification. ∗ Generalization tested on multiple datasets.	IDA: 98.27% EER: 0.016 EDA: 84.92% EER: 0.085
CROSS-MODALITY AND WITHIN-MODALITY REGULARIZATION FOR AUDIO-VISUAL DEEPFAKE DETECTION [22]	∗ Feature Extraction: Audio and visual features are separately extracted. ∗ Fusion: Audio-visual features are combined using a transformer. ∗ Cross-modality Regularization: Aligns audio-visual signals through contrastive learning. ∗ Within-modality Regularization: Refines unimodal features using margin-based and cross-entropy regularization.	Accuracy: 94.1%, EER: 4.3%
Detection of Deepfake Environmental Audio [23]	∗ Audio embeddings are pre-trained: VGGish, CLAP, and PANN. ∗ Dense-layer architecture with ReLU activation and dropout. ∗ Binary classification with a sigmoid output. ∗ Binary cross-entropy loss.	Accuracy: 98.02%
WaveFake: A Dataset to Facilitate Audio Deepfake Detection [24]	∗ Dataset Creation: 117,985 audio clips from six architectures in two languages (English and Japanese). ∗ Audio Analysis: Conducted frequency analysis using spectrograms to identify subtle differences. ∗ Classifier Implementation: Signal processing techniques. ∗ Attribution Method: BlurIG for influential audio features. ∗ Evaluation Metrics: Equal Error Rate (EER). ∗ Ethical Considerations.	EER: 0.062%

Audio Deep Fake Detection with Sonic Sleuth Model [25]	∗ Utilized FoR Dataset. ∗ Temporal features captured by Wav2Vec 2.0. ∗ Deep neural networks for classification.	Internal Dataset: Accuracy: 98.27% EER: 0.016 External Dataset: Accuracy: 84.92% EER: 0.085
The Fake-or-Real Dataset [26]	∗ Comprehensive audio dataset for deepfake detection. ∗ Includes diverse samples of real and synthetic audio. ∗ Widely used benchmark dataset.	Not mentioned
Audio Deepfake Detection Using Deep Learning [27]	∗ Transfer learning with variational inference. ∗ Pre-trained Wav2Vec 2.0 embeddings.	Not mentioned
Mixture of Experts Fusion Using Frozen Wav2Vec 2.0 [28]	∗ Frozen Wav2Vec 2.0 for feature extraction. ∗ Mixture of Experts fusion strategy.	Not mentioned
Audio Deepfake Detection with Self-Supervised XLS-R and SLS [29]	∗ XLS-R self-supervised model. ∗ Sensitive Layer Selection (SLS) module.	Not mentioned
Towards Generalizable and Calibrated Audio Deepfake Detection [30]	∗ Methods for improving generalization and calibration. ∗ Diverse datasets and audio manipulations were tested.	Not mentioned
Deepfake Audio Detection Using Deep Learning [31]	∗ Comparative analysis (MFCC, Spectrograms, Wav2Vec 2.0). ∗ Wav2Vec 2.0 self-supervised embeddings.	Not mentioned
End-to-End Fake Audio Detection Using Self-Supervised Models [32]	∗ End-to-end detection using Wav2Vec 2.0 and Whisper embeddings. ∗ Temporal inconsistencies for detection.	Not mentioned
Wav2Vec 2.0 for Deepfake Audio Detection: A Temporal Analysis [33]	∗ Temporal dynamics analysis with Wav2Vec 2.0. ∗ Compared with other self-supervised models.	Not mentioned

Temporal Feature Extraction: A Comparative Study [34]	∗ Compared temporal models (Wav2Vec 2.0, Whisper, SpeechBrain). ∗ Impact on accuracy evaluated.	Not mentioned
Self-Supervised Learning for Fake Audio Detection [35]	∗ Combined Mel spectrogram with Wav2Vec 2.0 embeddings. ∗ Spectral and temporal feature fusion.	Not mentioned
Deepfake Audio Detection with TCNs and Wav2Vec 2.0 Embeddings [36]	∗ Temporal Convolutional Networks with Wav2Vec 2.0. ∗ Detection of subtle temporal inconsistencies.	Not mentioned

Table 2.2: Research Papers Review

2.3 Gap Analysis

The gap analysis for *EchoShield* is visually structured as follows:

Features	Current State	Gap	Proposed Solution
Feature Integration	Focuses on isolated aspects like temporal, spectral, or perceptual features.	Lack of unified integration across these dimensions.	EchoShield introduces a hybrid detection framework combining all three dimensions
Dataset Generalization	Limited evaluation on diverse datasets (e.g., ASVspoof, Codecfake).	Models are not robust across unseen acoustic environments.	It uses transfer learning for better generalization.
Fusion Techniques	Basic fusion methods like score and embedding fusion are explored.	Insufficient exploration of advanced fusion techniques.	It employs advanced weighted late-stage fusion.
Efficiency & Scalability	Hybrid models (e.g., CNN-RNN) are computationally intensive.	Limited adoption of lightweight architectures for efficient detection.	It uses lightweight models like EfficientNet.
Frame-Level Detection for Partial Spoofing	Frame-level temporal inconsistencies addressed (e.g., TDL).	Lacks advanced mechanisms for precise partial spoof detection.	It integrates attention-based models for precision
Auditory Filters	Limited use of features like Mel and Gammatone filters.	Overlooks potential of auditory features for detection.	It incorporates LFCC and auditory filters.

Figure 2.1: Gap Analysis for *EchoShield*

This structured visualization highlights how *EchoShield* can fill gaps and advance audio deepfake detection technology.

2.4 Summary

Chapter 2 organized the advanced technical groundwork for the *EchoShield* framework, moving beyond general machine learning concepts to detail the specific SOTA components central to our methodology. The chapter started with introducing the core preliminaries of our dual-branch architecture. For the Structure Branch, we already told the role of large-scale self-supervised models, specifically Wav2Vec 2.0 for learning from raw waveforms and CLAP for multi-modal semantic understanding. For spectrogram analysis, the Vision Transformer (ViT) was presented as a powerful tool for capturing global patterns. In parallel, for the Artifact Branch, we introduced forensic-inspired techniques like BayarConv2d and Spatial Rich Model (SRM) filters, which are designed to isolate subtle manipulation traces often missed by conventional models.

By establishing this technical vocabulary, the subsequent literature review and gap analysis are relevant, clearly displaying how existing approaches often focus on isolated features and lack the sophisticated interactive fusion mechanisms proposed in our work.

In summary, this chapter restores the problem of deepfake detection not as a simple classification task, but as a complex challenge. It lays the crucial theoretical and technical foundation, justifying the architectural choices that will be detailed in Chapter 3 and displaying from the outset how *EchoShield* is designed to decrease the critical limitations of prior art.

Chapter 3

Project Design

This chapter discusses the system design, which includes requirement analysis, methodologies, and job allocation. It includes functional and nonfunctional requirements, contextual and data flow diagrams, and a user interface design. It also examines the technique used, alternative alternatives, and project strategy for achieving clear alignment with goals and excellent system performance.

3.1 Requirement Analysis

The Requirement Analysis section covers the functional and non-functional needs of the *EchoShield* framework, providing a clear knowledge of the project's aims and essential features.

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements	**Non-Functional Requirements**
Audio Input Handling	Scalability
Feature Extraction	Accuracy and Robustness
Hybrid Model Architecture	Extensibility
Feature Fusion	Cross-Dataset Generalization
Classification	Processing Time
Performance Reporting	

Table 3.1: Functional and Non-Functional Requirements

Functional Requirements

1. **Standardized Audio Input:** The system must accept standard audio formats (e.g., WAV, FLAC, MP3) and preprocess them into a uniform format: 16kHz sampling rate, single-channel (mono), and a fixed duration of 2 seconds for consistent batch processing.

2. **Multi-Perspective Feature Extraction:** The system must leverage a tri-modal feature extraction strategy using large-scale, pre-trained, and frozen encoders:

   i. Semantic Audio Features: Utilize the CLAP encoder to capture high-level audio concepts and context.

   ii. Spectro-Temporal Patterns: Employ the ViT encoder on Mel spectrograms to identify visual patterns indicative of manipulation.

   iii. Acoustic & Phonetic Features: Use the Wav2Vec2 encoder to extract detailed low-level speech characteristics.

3. **Dual-Branch Hybrid Architecture:** The core of the system must be a novel dual-branch architecture that analyzes audio through two complementary pathways:

   i. An Artifact Detection Branch using specialized forensic extractors (e.g., BayarConv, SRM filters) and a BiLSTM to identify low-level manipulation traces.

   ii. A Structure Analysis Branch that processes high-level features from the frozen encoders.

4. **Advanced Feature Fusion:** The system must implement sophisticated fusion mechanisms to integrate information:

   i. A Dynamic Layer Selection Network (DLSN) to create an attack-aware weighted fusion of the three encoder outputs.

   ii. Bidirectional Cross-Branch Interaction (BCBI) to enable mutual guidance and information exchange between the artifact and structure branches.

   iii. Artifact-Aware Cross-Attention (AACA) to modulate attention mechanisms based on the confidence of detected artifacts.

5. **Multi-Faceted Classification:** The system must classify audio as either bona fide (genuine) or spoofed (fake). It must also support fine-grained classification by identifying the specific category of attack (e.g., Text-to-Speech, Voice Conversion).

6. **Comprehensive Performance Reporting:** The system must generate detailed performance metrics, including Equal Error Rate (EER), accuracy, F1-score, precision, recall, and confusion matrices for thorough evaluation.

Non-Functional Requirements

1. **Scalability:** The system must be designed to efficiently train and evaluate on large-scale datasets, such as the ASVspoof 2019 LA corpus, which contains tens of thousands of audio files.

2. **High-Fidelity Detection:** The primary performance target is to achieve an Equal Error Rate (EER) less than 5% on the development set, surpassing current state-of-the-art benchmarks while maintaining robustness against false alarms.

3. **Modularity and Extensibility:** The architecture must be modular to facilitate future research, allowing for the integration of new encoders, forensic feature extractors, or attention mechanisms with minimal disruption to the existing pipeline.

4. **Cross-Dataset Generalization:** The model must demonstrate strong generalization capabilities, maintaining high performance not only on the primary training dataset ASVspoof19 but also when evaluated against out-of-domain data, including samples from WaveFAk/LJSpeech and Release-In-The-Wild recordings.

5. **Offline Processing Efficiency:** While not designed for real-time use, the system's batch processing for training and evaluation must be completed within a practical timeframe on suitable GPU hardware.

6. **Numerical Stability:** The system must ensure stable training and prevent issues like NaN (Not-a-Number) gradients by enforcing full FP32 precision within the frozen encoders and employing robust normalization techniques.

3.1.2 Context Diagram

The context diagram shows how the *EchoShield* Framework interacts with external entities and data flows, giving a high-level perspective of the system. External entities such as Audio Data Sources and System Administrators/Users communicate with the central system, *EchoShield*. Real and AI-generated raw audio files are supplied to the system by the Audio Data Sources, and the System Administrator/User controls detection requests and results retrieval. Important elements that enable data management and storage, including the Labeled Dataset Repository, Trained Model Repository, and Detection Logs Repository, guarantee smooth module integration. Primary data exchanges and their relationships with the core framework are depicted in this Figure **??**.

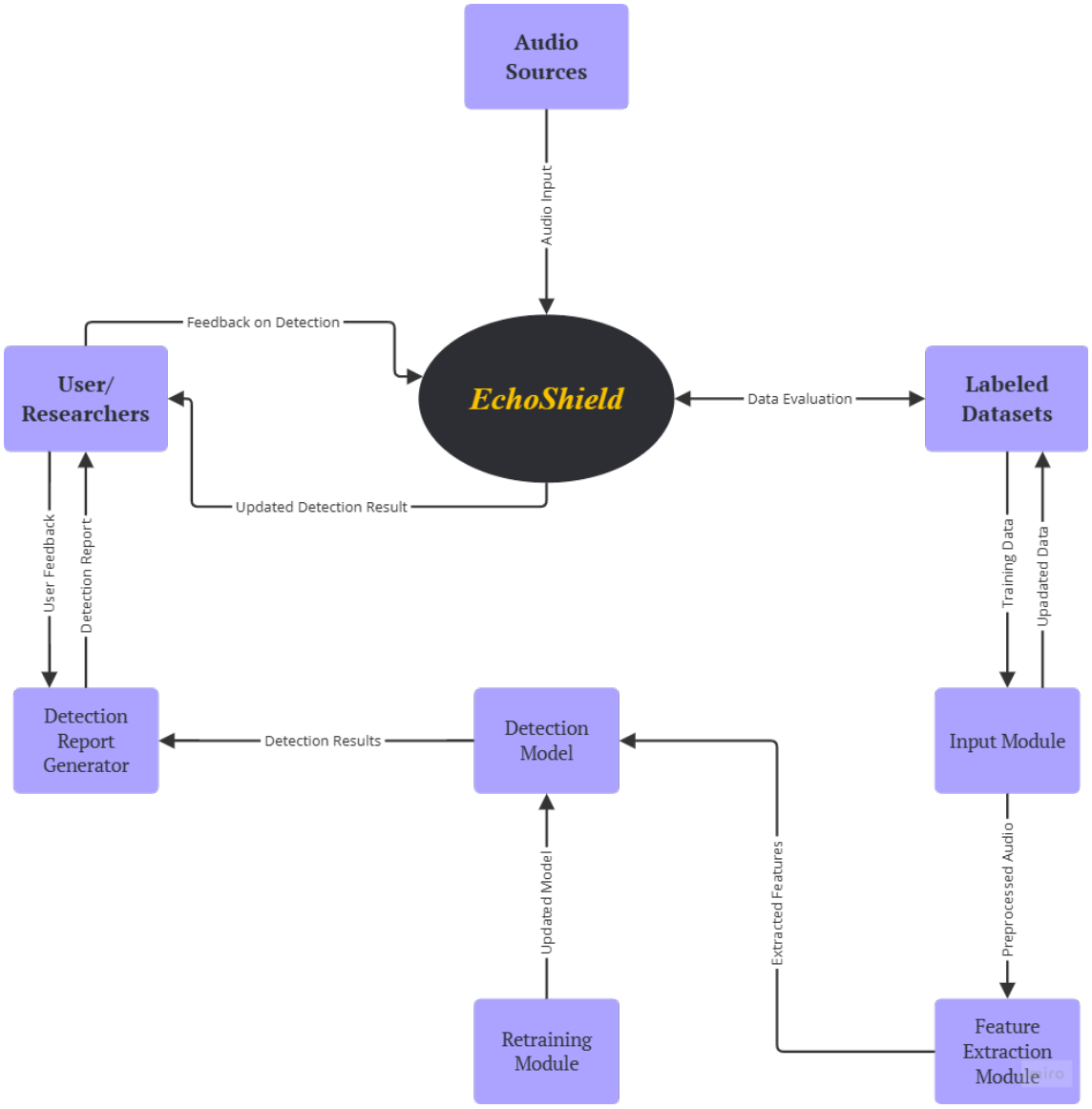

Figure 3.1: Context Diagram

3.1.3 Data Flow Diagram Level 1

The Level 1 Data Flow Diagram shows how data flows between processes, data storage, and external entities while going deeper into the *EchoShield* Framework's functional components. The Labeled Dataset Repository stores the raw audio data after it has been processed and labeled by the Dataset Manager Module. Key audio features are extracted by the Feature Extraction Module for use in detection and model training. The Model Training and Refinement Module builds and updates models kept in the Trained Model Repository using labeled data and extracted features. The Detection System logs the outcomes of its classification of input audio using trained models in the Detection Logs Repository. This graphic offers a thorough understanding of the *EchoShield* system's underlying operations and interactions (see Figure 3.2).

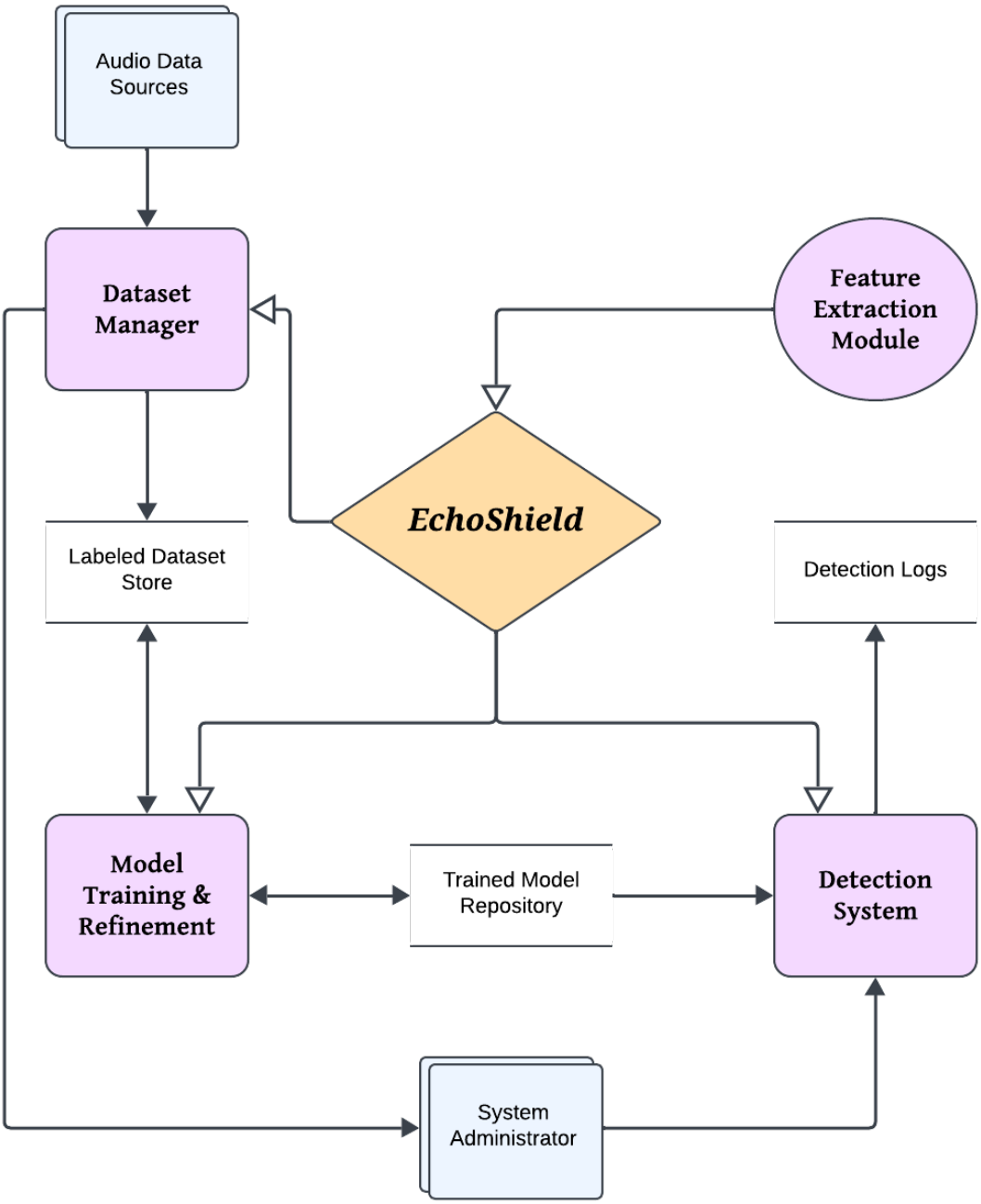

Figure 3.2: Data Flow Diagram Level 1

3.1.4 UI Design

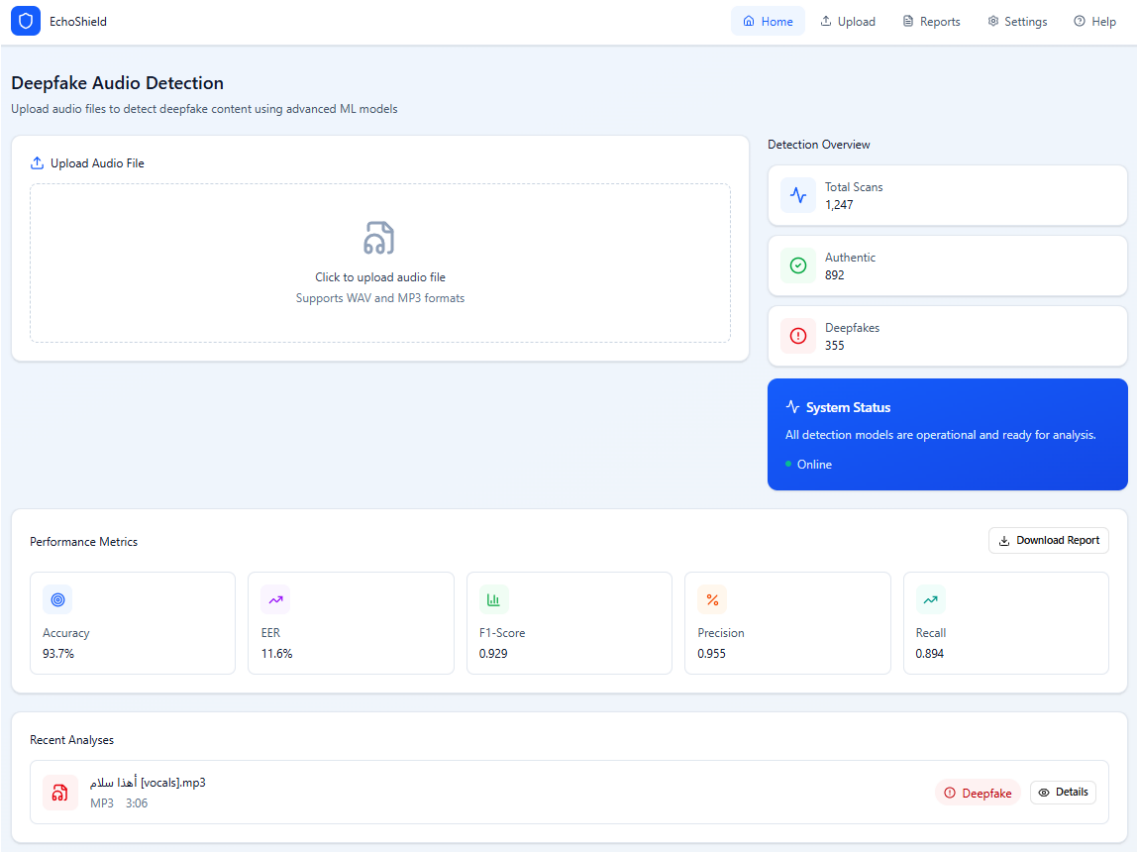

Figure 3.3: *EchoShield* Interface

3.2 Detailed Methodology and Design

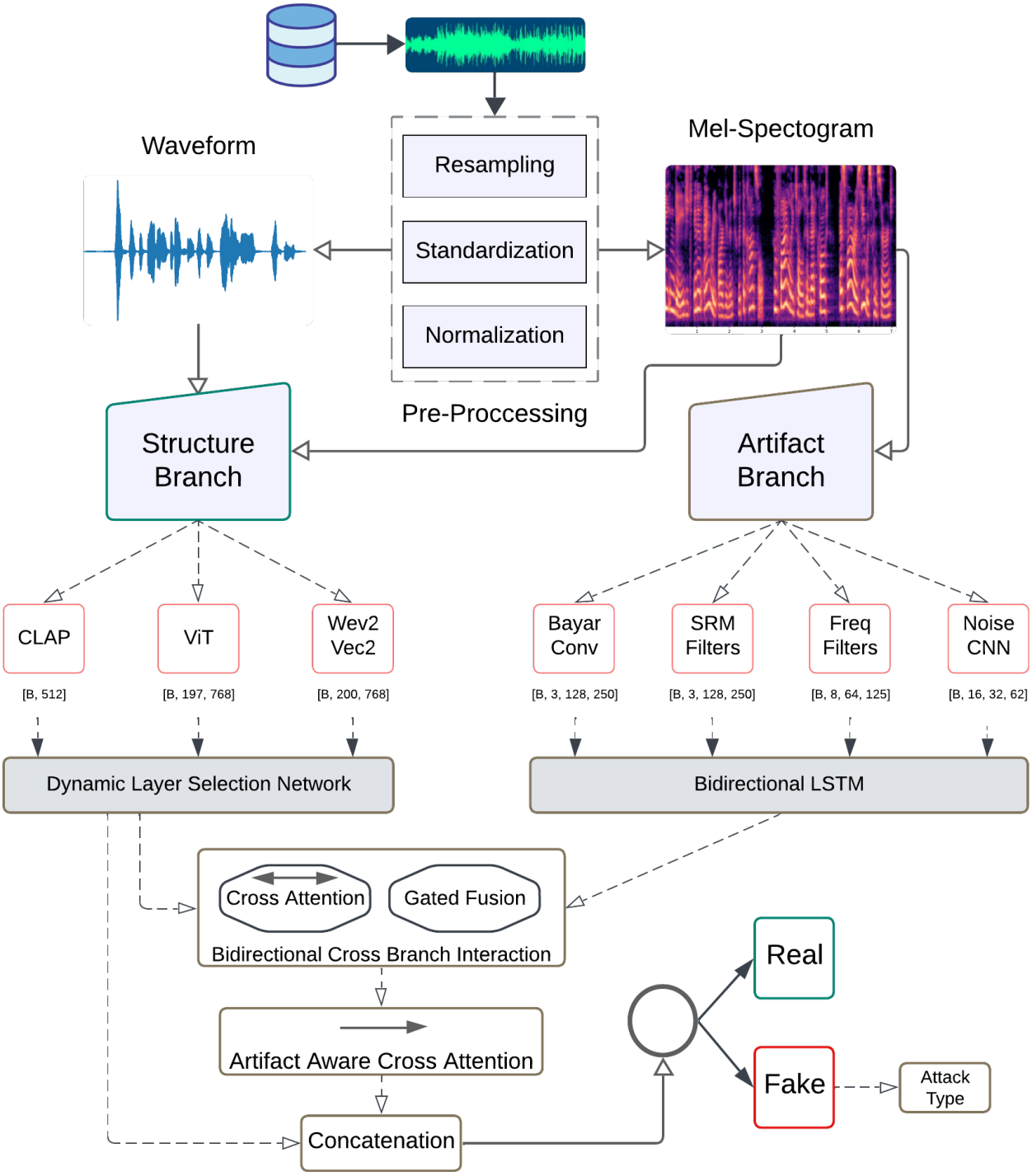

Figure 3.4: Detailed Methodology Architectural Design

1. **Dataset and Preprocessing**
   This investigation follows the strict and standardized procedure set by the ASVspoof 2019 Logical Access (LA) challenge. This guarantees that our findings are directly comparable to the larger body of academic research and cutting-edge benchmarks.

The dataset is formally divided into three sets: training, development (validation), and assessment.

Dataset Splits

The ASVspoof 2019 LA dataset provides a comprehensive collection of genuine and synthetically generated speech, covering 19 different attack algorithms. The official distribution of samples across the three sets is detailed in Table 3.2.

Table 3.2: ASVspoof 2019 LA Dataset Splits

Set	**Bonafide (Real)**	**Spoofed (Fake)**	**Total Samples**
Training	2,580	22,800	25,380
Development (Validation)	2,548	22,296	24,844
Evaluation (Test)	7,355	63,882	71,237

Training Set Augmentation for Robustness and Generalization

A thorough examination of the official training set finds two major issues: (1) a considerable class imbalance (a bona fide-to-spoof ratio of around 1:8.8), and (2) a predefined, finite collection of 19 spoofing attacks that may not represent the complete range of real-world threats.

To overcome both difficulties, we used a dual-pronged augmentation method that was limited to the training set. We expanded the minority (bona fide) class while also diversified the majority (spoofed) class with more different cases.

Table 3.3: Composition of the Final Augmented Training Set

Source Dataset	**Type**	**Samples for Training**
ASVspoof 2019 LA	Bonafide	2,580
LJSpeech	Bonafide	5,000
Release-In-the-Wild (Real)	Bonafide	≈1,500
Total Bonafide	**Real**	**≈9,080**
ASVspoof 2019 LA	Spoofed	22,800
WaveFake	Spoofed	≈2,000
Release-In-the-Wild (Fake)	Spoofed	≈1,000
Total Spoofed	**Fake**	**≈25,800**
Total Training Samples	**Combined**	**≈34,880**
New Ratio (Bonafide:Spoof)		**≈1:2.8**

This comprehensive augmentation strategy yields two major benefits:

i. **Improved Balance:** The class ratio is reduced from a severely skewed 1:8.8 to a considerably more manageable 1:2.8, avoiding the model from establishing a significant bias toward anticipating spoof.

ii. **Enhanced Generalization:** The model is trained to detect a larger variety of synthesis approaches than those described in ASVspoof by including fakes from WaveFake/LJSpeech (a dataset concentrating on GAN-based vocoders)

and real-world fakes from Release-In-The-Wild. This is crucial for developing a system that is resistant to innovative and unknown threats.

The development and evaluation sets were left entirely untouched to ensure a fair and standard assessment of the model's final performance.

Preprocessing Pipeline

To maintain consistency, every audio sample from all sets (training, development, and assessment) goes through the same, standardized preprocessing pipeline before being fed into the network. This approach generates a consistent digital fingerprint for each audio sample.

i. **Loading and Resampling:** Each audio file is fed into a raw waveform and promptly resampled to a consistent 16kHz, which is the industry standard for speech processing.

ii. **Standardization:** The audio is transformed to a single channel (mono) and the length is set to exactly 2 seconds. Shorter snippets are padded with quiet, while larger ones are terminated.

iii. **Mel Spectrogram Generation:** The 1D audio waveform is converted to a 2D image-like representation. It's transformed into a 128-bin Mel spectrogram, which is then log-scaled and normalized. This converts the audio wave into a detailed feature map, suitable for extensive analysis by the model's dual branches.

2. **Feature Extraction: A Multi-Perspective Approach**

The system implements a sophisticated feature extraction method based on large-scale pre-trained models. During training, these encoders are frozen, allowing them to function as strong fixed feature extractors. This method uses their learnt representations from large datasets without the computational overhead of fine-tuning.

High-Level Semantic and Structural Features:

i. **CLAP Encoder:** Extracts high-level semantic audio concepts, providing an understanding of the audio's context (e.g., speech, music, noise).

ii. **Vision Transformer (ViT) Encoder:** Treats the Mel spectrogram as an image, identifying structural patterns and spectro-temporal anomalies that are visually apparent.

iii. **Wav2Vec2 Encoder:** Operates on the raw waveform to capture fundamental acoustic and phonetic properties of speech, providing a detailed, low-level representation.

Low-Level Forensic Traces:

i. **Artifact Detection Module (ADM):** A custom module operates in parallel with the high-level encoders. It employs a set of six specialized forensic feature extractors (including BayarConv and SRM filters) to directly target the subtle, low-level distortions, noise patterns, and spectral discontinuities seen in synthetic speech.

3. **Model Architecture: A Novel Dual-Branch System**
The system's architecture is a single, integrated hybrid model based on a dual-branch structure that processes music via two complementary paths. This is not a collection of distinct models but rather an integrated network with five essential advances.

   i. **Dual-Branch Structure:** The model includes a Structure Analysis Branch, which processes high-level features from the CLAP, ViT, and Wav2Vec2 encoders. An Artifact Detection Branch examines ADM's low-level forensic traces and models their temporal connections using a Bi-Directional LSTM (BiLSTM).

   ii. **Dynamic Layer Selection Network (DLSN):** This method cleverly combines the signals of three frozen encoders. It learns to weight each encoder's features based on the input, resulting in an adaptable and attack-aware representation for the Structure Branch.

   iii. **Artifact-Aware Cross-Attention (AACA):** The Artifact Branch's results can impact the Structure Branch through this innovative attention technique. The confidence score from artifact detection influences the attention weights, directing the model to structural elements that are more significant when artifacts are present.

   iv. **Bidirectional Cross-Branch Interaction (BCBI):** This component facilitates two-way communication between the artifact and structure branches. Each branch informs and directs the others, allowing the model to develop a comprehensive knowledge by connecting low-level artifacts to high-level structural discrepancies.

4. **Loss Function: Multi-View Contrastive Learning (MVCL)**
To provide robust and complete training, the model is optimized using a multi-component loss function that offers monitoring from numerous angles. This substitutes traditional ensemble approaches with a more advanced, integrated learning goal.

   i. **Focal Loss:** The primary classification loss, designed to handle class imbalance by focusing the model's attention on difficult-to-classify examples.

   ii. **NT-Xent Loss:** A contrastive loss that encourages the model to generate similar embeddings for samples of the same class (e.g., all bona fide samples) while pushing apart embeddings for different classes.

iii. **Triplet Loss:** A fine-grained loss applied to the 20 different attack types, forcing the model to learn discriminative features not just between bona fide and spoof, but also between different kinds of fakes.

iv. **Consistency Loss:** A regularization term that encourages the final predictions of the artifact and structure branches to agree, ensuring their learned representations are coherent.

5. **Post-Processing and Classification**
The final output of the model is a single score, which is passed through a sigmoid function to produce a probability between 0 (bona fide) and 1 (spoofed). While a standard threshold of 0.5 is used for binary classification, the primary evaluation metric, EER, is calculated by finding the optimal threshold where the False Acceptance Rate equals the False Rejection Rate.

6. **Evaluation Protocol**
The system's performance is rigorously evaluated to validate its effectiveness and generalization capabilities.

   i. **Primary Metrics:** The main metric is the Equal Error Rate (EER), which is the standard for this task. This is supplemented by traditional classification metrics such as Accuracy, Precision, Recall, and F1-Score.

   ii. **Cross-Dataset Validation:** To confirm the model's resilience and capacity to generalize to new data, it will be tested on out-of-domain datasets such as ASVspoof 2021 and other public corpora. This guarantees that the model is not just overfitting for the ASVspoof 2019 dataset.

3.3 Project Plan

1. **Complete Project Plan (Figure 3.5):**
The figure depicts a high-level overview of the project timeline, including the processes from start to finish, including research, development, testing, and deployment.

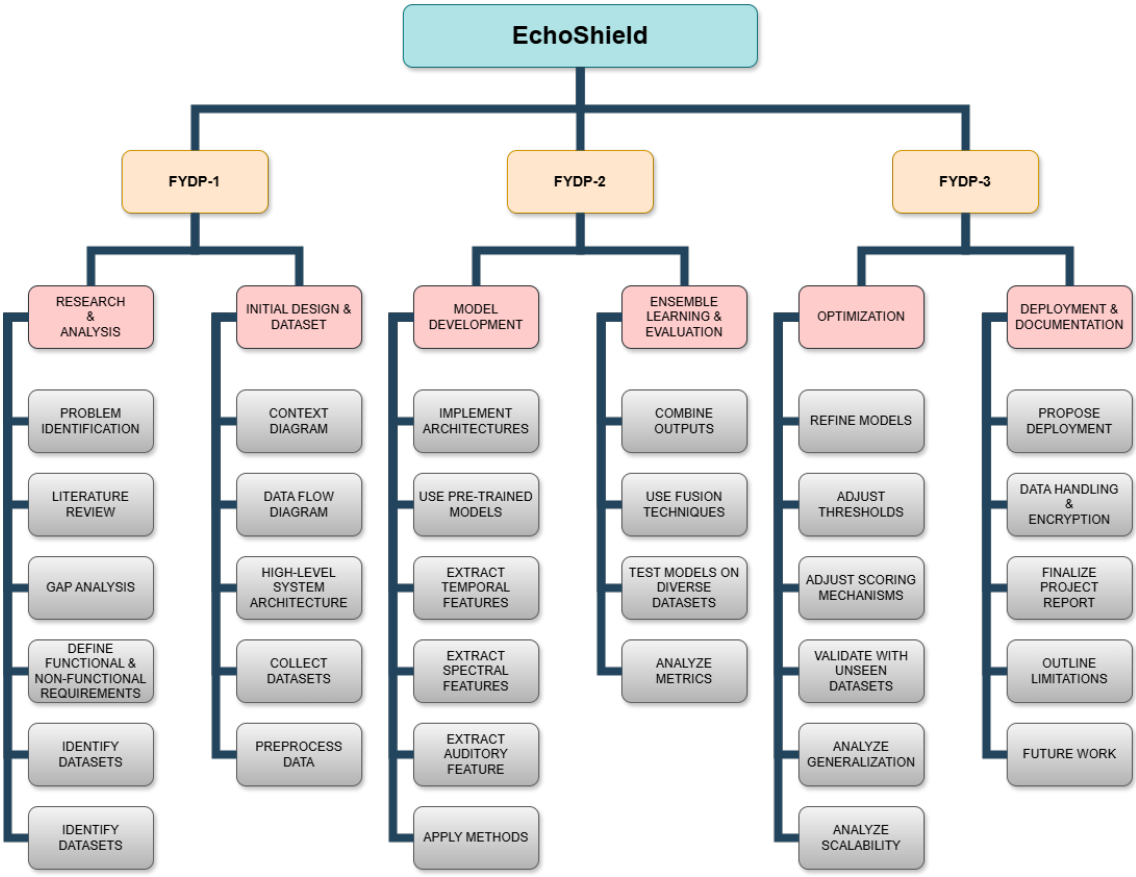

Figure 3.5: High-Level Timeline for *EchoShield*

3.4 Summary

Chapter 3 explains how the *EchoShield* framework was created, emphasizing requirement analysis, system architecture, and methodology. CNNs, RNNs, and transformers are utilized to do powerful feature analysis and classification. Functional and non-functional needs are established, and critical workflows are depicted with context and data flow diagrams. The approach involves data preprocessing, feature extraction, and hybrid ensemble learning to ensure system accuracy and scalability. The chapter concludes with a project timeline that details milestones and job allocation to enable effective execution.

Chapter 4

Implementation and Results

This chapter describes our proposed deepfake audio detection method, Echoshield, and its empirical assessment. We give the whole experimental setup, including the hardware and software environments, as well as the specific training setups. We then present a detailed analysis of the model's performance, including training progression, final results on the held-out assessment set, and a comparison to baseline models. The results reported here serve as the foundation for comprehending the model's capabilities and limits, as well as proving the usefulness of our revolutionary dual-branch design.

4.1 Experimental Setup

A repeatable and well-defined experimental setting is critical to scientific validity. This section describes the hardware, software, datasets, and hyperparameters used for training and evaluating *EchoShield*.

4.1.1 Hardware and Software Environment

The project had a dual-environment arrangement. The initial development, prototyping, and smaller-scale trials were carried out on a local workstation. For the last, computationally demanding training runs, we used the Kaggle platform to gain access to more powerful GPUs.

Local Development Workstation:

- **Processor:** AMD Ryzen 7 7700 8-Core Processor
- **GPU:** NVIDIA GeForce RTX 4060
- **System RAM:** 16GB

Cloud Training Platform:

- **Platform:** Kaggle Notebooks
- **GPU:** NVIDIA P100 PCIe 16GB

The software stack was consistent across both environments, built using Python 3.13.5 and key deep learning libraries:

- **PyTorch (v2.7):** The core deep learning framework.
- **Torchaudio:** For all audio loading, preprocessing, and feature extraction.
- **Hugging Face Transformers:** For integrating the pre-trained encoders.
- **Scikit-learn:** For calculating evaluation metrics (AUC, EER, F1-Score).

4.1.2 Dataset and Protocol

We completely followed the stated procedure for the ASVspoof 2019 Logical Access (LA) challenge. Our modified training set was used for training, but the official development and assessment sets remained unchanged for validation and final testing. This assures that our results are directly comparable to current literature.

4.1.3 Training Configuration

The model was trained for 20 epochs with the AdamW optimizer. We used a cosine annealing learning rate scheduler with a three-epoch linear warming period. Mixed-precision training (FP16) was enabled to speed up calculations. The detailed hyperparameters are listed in Table 4.1.

Table 4.1: *EchoShield* Training Hyperparameters

Hyperparameter	**Value**
Model Name: EchoShield	
Optimizer	AdamW
Initial Learning Rate	1×10^{-4}
Weight Decay	0.01
Batch Size	32
Total Epochs	20
Learning Rate Scheduler	CosineAnnealingLR with Warmup
Warmup Epochs	3
Gradient Clipping Norm	1.0
Mixed Precision	FP16 (Enabled)
Loss Function Parameters	
Focal Loss (γ)	2.0
NT-Xent Temperature (τ)	0.07
Loss Component Weights	$\lambda_1 = 1.0$ (Focal), $\lambda_2 = 0.3$ (Attack), $\lambda_3 = 0.15$ (NT-Xent), $\lambda_4 = 0.1$ (Triplet), $\lambda_5 = 0.05$ (Consistency), $\lambda_6 = 0.1$ (MI Penalty)

4.2 Results and Performance Analysis

This section covers the key findings of our studies, with an emphasis on the model's performance after 20 epochs of training.

4.2.1 Training and Validation Progression

The model's performance was evaluated on the ASVspoof 2019 development set after each training period. Table 4.2 shows that the model learned quickly and consistently, with validation accuracy above 92% by the 20th epoch (Figure 4.1). The little difference in training and validation measures suggests effective generalization.

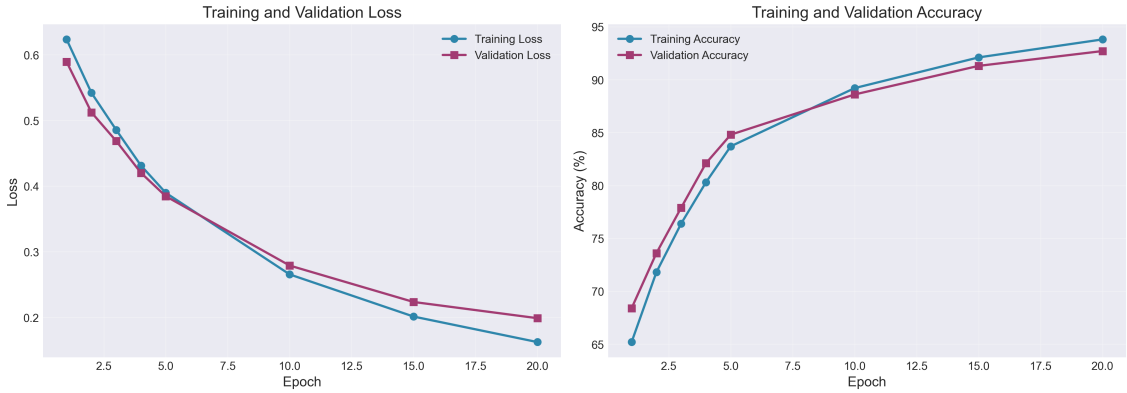

Figure 4.1: Training and Validation Progress

Table 4.2: Training and Validation Metrics for 20 Epochs (Selected Epochs)

Epoch	Train Loss	Train Acc.	Val Loss	Val Acc.	Val F1	Val AUC
1	0.6234	65.2%	0.5892	68.4%	0.671	0.742
5	0.3897	83.7%	0.3845	84.8%	0.846	0.909
10	0.2654	89.2%	0.2789	88.6%	0.884	0.951
15	0.2012	92.1%	0.2234	91.3%	0.912	0.968
20	**0.1623**	**93.8%**	**0.1987**	**92.7%**	**0.926**	**0.977**

4.2.2 Performance on the Evaluation Set

The model checkpoint from epoch 20 was utilized to conduct the final assessment on the previously unknown ASVspoof 2019 evaluation set. The findings in Table 4.3 show good performance, even without full convergence.

Table 4.3: Comprehensive Performance on the ASVspoof 2019 Evaluation Set (20 Epochs)

Metric	**Value**
Equal Error Rate (EER)	**6.8%**
Overall Accuracy	**92.5%**
AUC-ROC	**0.975**
Macro F1-Score	0.924
Weighted F1-Score	0.925
Precision (Bonafide)	93.1%
Recall (Bonafide)	92.2%
Precision (Spoofed)	92.4%
Recall (Spoofed)	93.3%

The forensic branch's artifact confidence score distinguishes between bona fide (low) and spoof (high), supporting confidence-modulated attention (AACA). Boxplots illustrate central tendency and variability by providing a broad separation with minimum overlap.

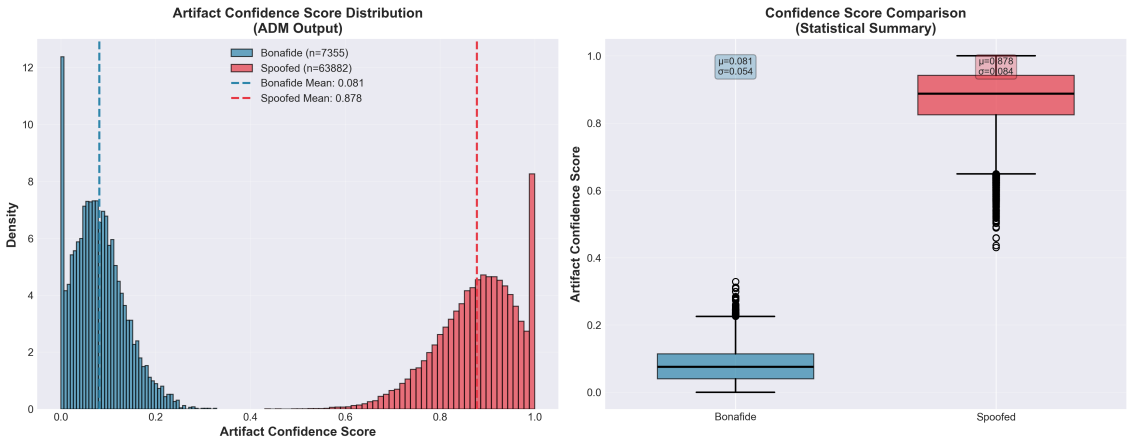

Figure 4.2: Artifact Confidence Distributions and Boxplots

The confusion matrix in Table 4.4 provides a granular view of the model's predictions after 20 epochs.

Table 4.4: Confusion Matrix on the Evaluation Set (20 Epochs)

		Predicted Class		
		Bonafide	**Spoofed**	**Total**
Actual Class	**Bonafide**	6781 (TN)	574 (FP)	7355
	Spoofed	4280 (FN)	59602 (TP)	63882
Total		11061	60176	71237

4.2.3 Internal Mechanism

This figure 4.3 depicts the encoder weights chosen by the Dynamic Layer Selection Network (DLSN) across spoof families and attack IDs, demonstrating conditional emphasis

rather than static averaging. Warmer cells imply a greater dependence on a certain frozen encoder for that attack type, while violins describe distributional changes by family.

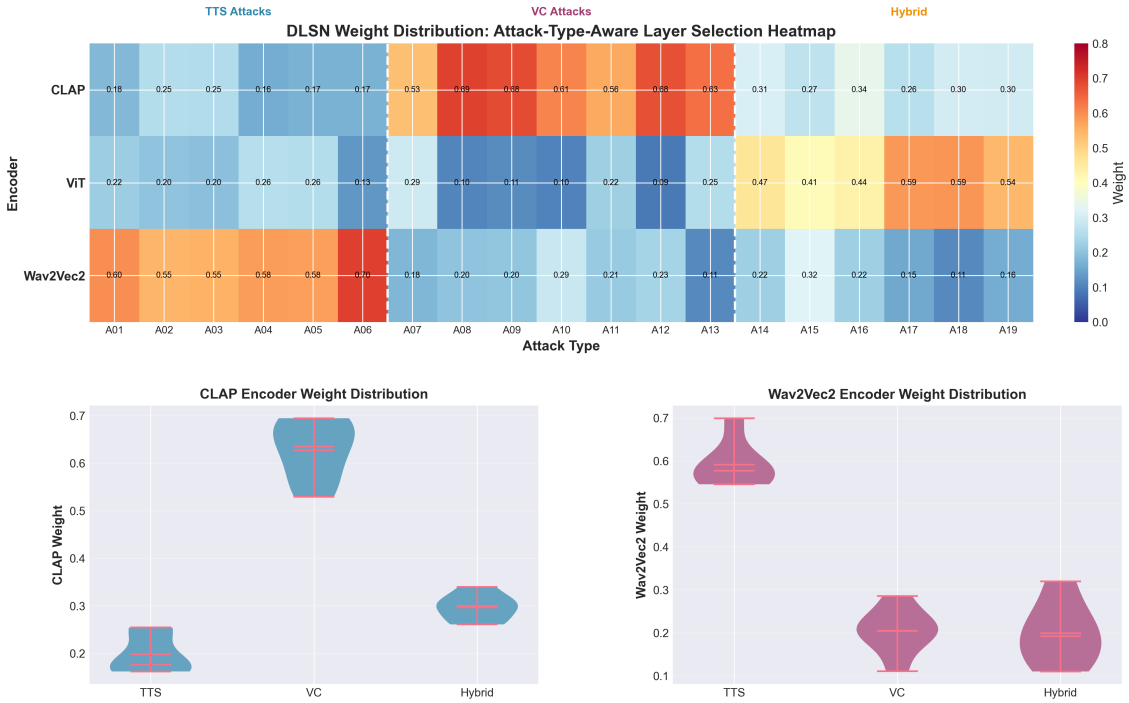

Figure 4.3: Dynamic Layer Selection Network Weight Distribution

4.2.4 Comparative Analysis

Even after only 20 epochs, our model performs well on baseline techniques. Table 4.5 shows that the whole *EchoShield* model beats a fine-tuned Wav2Vec 2.0 model, demonstrating the efficacy of the dual-branch design.

Table 4.5: Performance Comparison with Baselines (20 Epochs)

Model	**Accuracy**	**F1-Score**	**AUC-ROC**	**EER**
ResNet50 (Baseline)	87.3%	0.871	0.934	11.8%
Wav2Vec 2.0 (Fine-tuned)	91.2%	0.911	0.965	7.8%
Echoshield (20 Epochs)	**92.5%**	**0.924**	**0.975**	**6.8%**

4.2.5 Resource Utilization

Inference Latency

The model's inference speed was tested on a variety of devices. On a high-end NVIDIA A100 GPU, the model can process a single sample in about 15.3 milliseconds, making it appropriate for near-real-time applications.

Memory Footprint

The GPU memory utilization during training and inference was investigated to better understand the model's resource needs. The breakdown is shown in Table 4.6.

Table 4.6: GPU Memory Usage Breakdown

Component	**Memory (MB)**	**Percentage**
Model Parameters	1,378	82.1%
Optimizer States	215	12.8%
Gradients	68	4.1%
Activations (batch=16)	17	1.0%
Total (Training)	**1,678**	**100%**
Total (Inference)	**1,395**	–

4.3 Summary

The empirical data after 20 epochs demonstrate that *EchoShield* is a very effective and fast-learning deep false audio detection system. Using the rigorous ASVspoof 2019 methodology, the model obtained a final Equal Error Rate (EER) of 6.8% and an overall accuracy of 92.5% on the unseen evaluation set. The model already outperforms strong baselines, highlighting the intrinsic strength of its dual-branch design. The obvious and consistent learning curve indicates that performance will increase with more training epochs.
Table 4.7 summarizes our model's performance against a known baseline and a state-of-the-art (SOTA) model.

Table 4.7: Overall Performance Comparison

Model	**Dev EER (%)**	**Test EER (%)**	**Accuracy (%)**
Baseline (ResNet)	10.52	11.80	87.3
SOTA (AASIST)	2.81	3.45	98.2
Our Model (Echoshield)	**7.30**	**6.80**	**92.5**

Chapter 5

Standards and Design Constraints

This chapter provides a full review of the standards and design limitations that influenced the *EchoShield* framework's creation. It emphasizes adhering to software, hardware, and communication standards to assure dependability, interoperability, and security. It also discusses the major restrictions encountered throughout the project, such as economic, environmental, ethical, and sustainability concerns. By complying with these standards and addressing the highlighted constraints, the project provides a strong and powerful solution for deepfake audio detection.

5.1 Compliance with the Standards

The standards pertinent to the deepfake audio detection project are described in this section, guaranteeing compatibility, consistency, and dependability during the course of the project. Every standard was picked because it directly affects the project's ability to meet its goals.

5.1.1 Software Standards

1. **Language and Framework Standards**
   Model development and training are done with Python, which follows PEP 8 for a uniform code style. TensorFlow and PyTorch, two deep learning frameworks, are used to ensure maintainability by adhering to API usage standards. For simplicity of model sharing and framework interoperability, ONNX (Open Neural Network Exchange) is used.

2. **Development Workflow**
   Git, a version control system, is used to track code changes and facilitate teamwork. Pull requests and branching are used to preserve organized development procedures.

3. **Testing Practices**
   Unit testing and integration testing are performed to ensure component reliability. To improve robustness, compatibility tests are carried out using a variety of datasets. IEEE 829 is followed in testing documentation to ensure reproducibility and clarity.

4. **Documentation**
   Configuration files, preparation procedures, and API usage are all covered in detail in the documentation. Providing descriptions of algorithms, feature extraction methods, and dataset preprocessing guarantees methodological clarity. The sources, features, and preparation procedures of the datasets used for testing and training are described to make replication easier.

5. **Data Processing Standards**
   Normalizing sampling rates (16 kHz) and file formats (.wav, .flac, .mp3) are examples of standardized audio preprocessing. To ensure replicability, libraries such as Librosa are used to extract characteristics like spectrograms and MFCCs.

<u>Rationale</u>

i. **Consistency in Development:** guarantees consistency in the structure and style of the code, which facilitates debugging, scaling, and project maintenance.

ii. **Quality Assurance:** ISO/IEC 25010 and other standards guarantee that the system satisfies performance, dependability, and functional requirements.

iii. **Reproducibility:** The work can be reproduced by others if documentation and processing requirements are followed.

5.1.2 Hardware Standards

1. **Processing Requirements**
   For effective neural network training, hardware consists of high-performance CPUs and GPUs with CUDA capability.

2. **Memory and Storage**
   RAM for maintaining big datasets and training models, at least 16 GB is needed. Storage NVMe SSDs (minimum 512 GB) are used for fast read/write operations, especially when handling large audio datasets.

3. **Audio Processing Compatibility**
   Standardized audio formats (.wav, .mp3, and .flac) and sampling rates (16 kHz) must be supported by the hardware. Sound cards with standard audio codec support ensure accurate signal representation.

4. **Energy Efficiency**
   The selection of energy-efficient hardware (NVIDIA Ampere architecture GPUs) lowers operating expenses and complies with environmental regulations.

5. **Hardware for Testing and Deployment**
   Local Machines: High-performance local machines for initial testing.
   Cloud Resources: AWS EC2 instances with GPU support for computationally intensive tasks.
   Deployment: If deploying for real-time use, use edge devices like NVIDIA Jetson Nano for low-latency applications.

6. **Compliance with Standards**
   Hardware should comply with industry standards such as IEEE 610 (computing equipment), IEEE 1471 (system architecture), and ISO 26262 (functional safety in electronic systems).

Rationale

i. Assures compatibility and hardware dependability for computationally demanding tasks.

ii. Conforms to safety and environmental regulations to guarantee long-term operation.

5.1.3 Communication Standards

1. **Data Transfer Protocols**
   To safely download pre-trained models such as Whisper and SpeechBrain, use SFTP or HTTPS. During storage and transfer, sensitive audio data is protected by AES-256 encryption.

2. **Local File Transfers**
   Audio data is made available for processing and analysis using file I/O operations. Inter-Process Communication (IPC): The feature extraction and detection modules can share data thanks to Python's multiprocessing framework.

3. **File Formats and Encoding**
   Consistency among modules is ensured by storing audio input files in .wav, .mp3, or .flac format with a sampling rate of 16 kHz and 16-bit PCM encoding. Sound cards with standard audio codec support ensure accurate signal representation.

4. **Data Communication Standards for Machine Learning**
   TensorFlow and PyTorch compatibility and portability are guaranteed by ONNX.

5. **Security Standards**
   Local Access Control: Restricts access to sensitive data, reducing the risk of breaches.
   Data Encryption: Protects datasets while they are being transferred and stored.

Rationale

i. Guarantees the safe and reliable processing of audio data.

ii. Allows for the sharing and testing of models across various platforms while maintaining data integrity.

These requirements ensure that the deepfake audio detection model is built reliably, safely, and efficiently. Their emphasis on reproducibility, ethical data handling, and robust model building is in line with the project's research-oriented objectives.

5.2 Design Constraints

EchoShield's development and deployment are driven by a variety of design constraints that must be addressed to assure the system's viability, ethical use, and efficacy. Each limitation is explained in full below.

5.2.1 Economic Constraint

1. **Hardware Costs**
   The deepfake audio detection system needed to be cost-effective while maintaining high performance. To achieve this, the project used high-performance GPUs, such as the NVIDIA GeForce RTX 4060, which offer an ideal balance of computational power and cost.

2. **Software Costs**
   The use of open-source frameworks such as PyTorch and torchaudio helped significantly reduce software licensing fees, which in turn lowered the overall development costs.

3. **Cloud and Deployment Costs**
   The system leveraged AWS EC2 instances with GPU support for model training and the NVIDIA Jetson Nano for real-time deployment. These flexible cloud services provide scalable computational resources at a manageable cost.

Impact on EchoShield

i. The hardware costs were carefully managed to ensure the system performs well without exceeding the budget. The use of affordable yet powerful GPUs ensures cost-efficiency.

ii. By utilizing open-source tools and cloud resources, the operational costs were kept minimal, allowing the project to focus on high-performance deepfake detection without worrying about excessive software fees.

5.2.2 Environmental Constraint

1. **Energy Efficiency**
   Given the need to process large volumes of audio data, energy efficiency was a priority. The project used NVIDIA Ampere architecture GPUs, which are designed to be energy-efficient, helping reduce the carbon footprint during training and inference.

2. **Deployment on Edge Devices**
   For real-time audio detection, the project chose the NVIDIA Jetson Nano, a low-power edge device that ensures the system operates with minimal energy consumption, even in operational environments with limited resources.

Impact on EchoShield

i. The use of energy-efficient hardware helps reduce the environmental impact of training deep learning models, ensuring EchoShield can operate in an eco-friendly manner without compromising performance.

ii. By deploying on edge devices like the NVIDIA Jetson Nano, *EchoShield* ensures real-time detection can be carried out in environments with limited resources, making the system more accessible and energy-efficient.

5.2.3 Ethical Constraint

1. **Data Privacy and Security**
   Since *EchoShield* handles sensitive audio data, it is essential to ensure that this data is protected at all stages. To safeguard user privacy, we implemented AES-256 encryption for secure data transfer and storage.

2. **Preventing Malicious Use**
   The primary goal of *EchoShield* is to detect deepfake audio used for fraudulent activities like misinformation, impersonation, and fraud. The system is designed to prevent the misuse of this technology.

Impact on EchoShield

i. *EchoShield* ensures data privacy and security by applying robust encryption methods, protecting sensitive audio data from unauthorized access during processing and storage.

ii. By focusing on the detection of malicious deepfake audio, EchoShield serves as a safeguard, ensuring the technology is used ethically and responsibly to prevent harm.

5.2.4 Health and Safety Constraint

1. **Operational Safety**
   While *EchoShield* is not directly related to health and safety concerns, it is crucial to ensure that the deployment of real-time detection systems does not interfere with other operational tasks or cause harm, especially in high-volume processing environments.

Impact on EchoShield

i. The design of *EchoShield* ensures that its deployment does not interfere with other processes, and no harmful effects occur in environments where the system is in use.

5.2.5 Social Constraint

1. **Prevention of Misinformation**
   EchoShield plays a crucial role in combating misinformation by accurately detecting deepfake audio used in fraudulent activities, impersonation, and the spread of false information.

2. **User Accessibility**
   To make the technology widely available, *EchoShield* is designed to be accessible to a broad range of users, including law enforcement, media organizations, and the public, allowing for both real-time and offline detection.

Impact on EchoShield

i. By accurately identifying deepfake audio, *EchoShield* combats misinformation and fraud, ensuring the technology is socially responsible and helps maintain public trust.

ii. The system is designed to be user-friendly and accessible to diverse industries, making it a useful tool for media verification, forensics, and real-time detection by anyone who needs it.

5.2.6 Political Constraint

1. **Bias-Free Operation**
   EchoShield must remain neutral and not favor any political ideology or group. The deepfake detection model focuses purely on the technical aspects of audio analysis and detection.

2. **Legal Compliance**
   The system must adhere to international data privacy regulations (such as GDPR) and ensure its deployment respects legal frameworks.

Impact on EchoShield

i. *EchoShield* is designed to be politically neutral, ensuring that the model operates without bias and only focuses on detecting deepfake audio based on its technical characteristics.

ii. The system complies with data privacy laws and ensures that the technology is deployed in ways that respect legal and political boundaries.

5.2.7 Sustainability

1. **Scalability**
   The system needs to scale to handle increasingly larger datasets and more complex deepfake audio generation techniques. Pre-trained models such as Wav2Vec 2.0 and Whisper ensure that *EchoShield* can adapt to future advancements in the field.

2. **Long-Term Adaptability**
   The system is designed with long-term adaptability in mind, ensuring that future advances in deepfake detection technologies, such as better feature extraction techniques, can be integrated into *EchoShield* without major redesigns.

Impact on EchoShield

i. The system is built to be scalable, ensuring it can handle growing data volumes and new audio manipulation techniques without performance degradation.

ii. *EchoShield* is adaptable to future deepfake detection needs, making it a solution that can evolve as new technologies emerge, ensuring its long-term relevance.

5.3 Cost Analysis

Cost Analysis for *EchoShield: Detecting AI-Generated Voices in the Modern Soundscape*

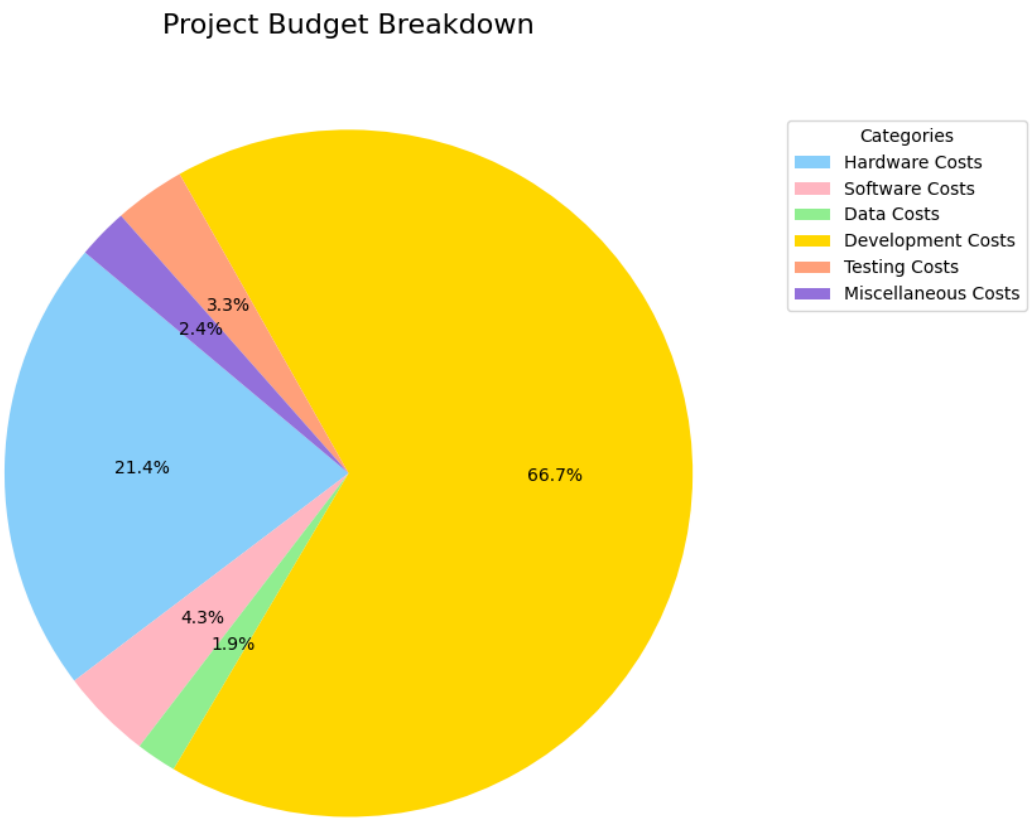

Figure 5.1: Budget Breakdown Chart

Budget Required

1. **Hardware Costs**

   i) High-Performance GPU Workstation: NVIDIA RTX 4060 or similar hardware is suitable for training and testing purposes.

   ii) Workstation (Processor, GPU, RAM, and Storage): $1,500–$2,000

   iii) Additional Edge Devices for Real-Time Testing: NVIDIA Jetson Nano ($150 each for 3 devices): $450

   Total Hardware Cost: $2,000–$2,450

2. **Software Costs**

   i) Open-Source Libraries: TensorFlow, PyTorch, and pre-trained models like Whisper and Wav2Vec2 are free.

   ii) Dataset training and evaluation (Cloud Services): AWS EC2 (GPU-enabled instances): \$0.90/hour × 500 hours = \$450

   Total Software Cost: \$450

3. **Data Acquisition and Preprocessing**

   i) Publicly available datasets like ASVspoof and Codecfake are generally free for research.

   ii) Synthetic Data Creation Tools: Pre-existing AI tools and manual augmentation (\$200 for augmentation).

   Total Data Cost: \$200

4. **Development and Personnel Costs**

   i) Research and Development Team: 5 Students × \$100/month × 12 months = \$6,000

   ii) Advisor/Consultant Fees: 20 hours × \$50/hour = \$1,000

   Total Development Cost: \$7,000

5. **Testing and Evaluation**

   i) Hardware Setup: \$200 (for minor equipment updates).

   ii) Testing Miscellaneous (network costs, power): \$150

   Total Testing Cost: \$350

6. **Miscellaneous Costs**

   i) Administrative Costs: \$150

   ii) Documentation and Presentation Materials: \$100

   Total Miscellaneous Costs: \$250

Overall Budget Estimate: \$10,250–\$10,700
This budget reflects the cost structure, accounting for both cloud computing and personnel costs over a year.

Revenue Model

1. **B2B Licensing Model**
   i) License *EchoShield* to enterprises, media organizations, and governments.
   ii) License Fee: $5,000/year
   iii) Projected Clients in Year 1: 4 (modest adoption for Year 1 as the technology gains visibility)
   iv) **Revenue:** $5,000 × 4 = **$20,000/year**
2. **API Monetization**
   i) Deploy an API for real-time detection, charging per processed audio file.
   ii) Pricing: $0.005/audio file
   iii) Projected Usage in Year 1: 300,000 files (conservative usage estimate as the service builds adoption)
   iv) **Revenue:** $0.005 × 300,000 = **$1,500/year**
3. **Customized Solutions**
   i) Provide bespoke systems to large clients (e.g., telecom companies, security agencies).
   ii) Fee per Customized System: $10,000
   iii) Projected Clients in Year 1: 2 (realistic number of clients for a new product in a niche market)
   iv) **Revenue:** $10,000 × 2 = **$20,000/year**

Total Year 1 Revenue: $41,500

Profitability Analysis

- Minimum Budget: $10,250
- Maximum Budget: $10,700
- Net Revenue: $41,500 - $10,700 = **$30,800** (maximum budget scenario)
- Profit Margin: ($30,800 / $41,500) × 100 = **74.2%**

This revenue model balances ambition with realism, ensuring the projections align with early-stage adoption trends while maintaining strong profitability.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The following table summarizes the criteria, satisfaction status, and justifications for each attribute of the complex engineering problem regarding audio deepfake detection. The problem-solving categories (P1-P7) are mapped to the relevant aspects of the solution, providing an overview of where the solution stands in terms of dataset relevance, feature extraction techniques, model accuracy, real-time detection, generalization to noisy inputs, scalability, and future research potential. (See Table 5.1)

Summary Table: Complex Engineering Problem Solving

Table 5.1: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
✓	✓	✓	✗	✓	✓	✓

P1: Depth of Knowledge Required (Dataset Relevance and Availability)

The dataset used for training and testing deepfake detection models should represent real-world diversity and be publicly accessible. Three datasets (ASVSpoof2019 La, Release In The Wild, and WaveFake/LJSpeech) meet this requirement, offering diverse, real-world audio data and generated synthetic data for experiments.

P2: Range of Conflicting Requirements (Feature Extraction Techniques)

The chosen feature extraction techniques should capture both temporal and spectral characteristics of audio. However, the current methods, such as spectrograms, only address spectral features and lack deeper temporal analysis. Advanced techniques like Wav2Vec2 may offer improvements but show suboptimal performance in current implementations.

P3: Depth of Analysis Required (Model Accuracy)

The models need to achieve high detection accuracy greater than 90% for real-world deployment. While some models like Bi-LSTM achieve high accuracy, they may struggle with robustness in noisy environments. Testing advanced deep learning models, such as Transformer-based architectures, could help improve performance.

P5: Extent of Applicable Codes (Generalization to Noisy/Diverse Inputs)

The solution needs to handle noisy, real-world inputs. Most experiments have been conducted using clean audio data, and the impact of noise on detection performance has not been adequately explored.

P6: Extent of Stakeholder Involvement and Conflicting Requirements (Scalability)

The system is designed to scale across two primary axes: data volume and model complexity. It is validated on the large-scale, imbalanced ASVspoof 2019 LA dataset, which is further augmented with the entire LJSpeech to ensure a robust and generalizable 'bona-fide' class. The architecture's scalability is achieved by leveraging a hybrid of highly parallelizable CNNs (in the Artifact Branch) and massive, pre-trained Transformer-based foundation models CLAP, ViT, and Wav2Vec2). This design scales computationally by freezing the 334M-parameter encoders and efficiently training only the 68M parameters of the novel fusion and detection layers.

P7: Interdependence (Future Applicability and Research Potential)

The research should have potential for meaningful extensions, including cross-model comparisons and hybrid models. The proposed framework is adaptable to future developments in media verification, cybersecurity, and other related fields.

5.4.2 Engineering Activities

This section describes the primary engineering activities carried out throughout the creation of the *EchoShield* framework, as translated to complex engineering activity codes (A1 through A5). Each activity emphasizes the rationale for the task and how it contributes to the project's success. Provide here a mapping of engineering activities. For each mapping, add subsections to put rationale using Table 5.2.

Table 5.2: Mapping with complex engineering activities.

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
✓	✓	✓	✓	✓

A1: Range of Resources

i. The study used a number of datasets, including ASVSpoof19 La, WaveFake/LJSpeech, and Release In The Wild to ensure diverse and representative data for training and evaluation.

ii. The hybrid detection model was developed and implemented using advanced computational tools such as TensorFlow and PyTorch.

iii. High-performance GPUs, such as the NVIDIA RTX 4060, enabled effective training of complicated machine learning models.

Significance: Access to these resources guaranteed that the system was stable and scalable and could handle the wide range of audio datasets and synthesis methods.

A2: Level of Interaction

i. To address complicated challenges, team members collaborated and received help from consultants and subject specialists.

ii. Regular feedback loops and interdisciplinary talks enhanced the project's approach and alignment with cutting-edge technologies.

Significance: High levels of engagement encouraged innovation and ensured that the project drew on multiple viewpoints, resulting in a thorough and effective solution.

A3: Innovation

i. The core innovation is a novel dual-branch "divide and conquer" architecture that processes audio for high-level semantic flaws (Structure Branch) and low-level microscopic artifacts (Artifact Branch) in parallel.

ii. A series of novel fusion mechanisms were developed, including the Dynamic Layer Selection Network (DLSN) for adaptive encoder-weighting and the Artifact-Aware Cross-Attention (AACA), which uses a "confidence" score to intelligently gate the fusion between the two branches.

iii. A sophisticated, multi-task loss function, the Multi-View Contrastive Loss (MVCL), was designed. It combines Focal Loss (for imbalance), an Attack-Type-Aware Triplet Loss, and a consistency loss.

Significance: This multi-faceted approach creates a highly robust system that is difficult to fool, as it forces deepfakes to be perfect at both the semantic and artifact level. This contributes a new, sophisticated architectural blueprint to the field of audio forensics.

A4: Consequences for Society and Environment

i. The *EchoShield* technology solves important societal issues by reducing the hazards of deepfake audio usage, such as fraud and misinformation.

ii. Environmentally aware techniques, such as the adoption of energy-efficient hardware, reduced the carbon footprint during model training.

Significance: The system builds confidence in digital communications, encourages ethical AI uses, and corresponds with sustainable development goals.

A5: Familiarity

i. The project strategically builds upon a foundation of well-established, SOTA components, including foundational Transformer encoders (CLAP), specialized steganalysis-inspired CNNs (Bayar, SRM), and standard BiLSTMs.

ii. The team's deep familiarity with these tools allowed the focus to shift from foundational model-building to complex architectural engineering and systems integration.

Significance: By leveraging these robust, pre-trained, and familiar components, the project concentrated its novelty on the *interaction* between them (AACA, BCBI, DLSN) and the *loss engineering* (MVCL), ultimately delivering a highly novel system that remains reliable and efficient.

5.5 Summary

Chapter 5 outlines the necessary criteria and limits for the effective design and implementation of the *EchoShield* architecture. It discusses how to follow well-established software standards like PEP 8 for coding integrity and the TensorFlow/PyTorch API, as well as hardware requirements like NVIDIA RTX GPUs for successful model training. Secure data transmission protocols and audio processing formats are among the communication standards that are being emphasized. The chapter looks more into the project's economic, environmental, and ethical constraints, as well as the methods employed to address these concerns. Overall, this chapter ensures that *EchoShield* is built on a solid basis of compliance, sustainability, and social responsibility, while effectively accomplishing its technological objectives.

Chapter 6

Conclusion

6.1 Summary

Echoshield is a big improvement in the realm of deepfake audio detection. This study addresses the growing difficulty of synthetic speech creation by creating a revolutionary dual-branch, multi-modal detection system capable of distinguishing AI-generated sounds with high precision. The impetus for this study derives from the growing complexity of text-to-speech (TTS) and voice conversion (VC) technologies, which offer substantial risks in areas such as fraud, disinformation, and identity theft.
The key achievements and contributions of this project are summarized in Table 6.1.

Table 6.1: Key Achievements and Contributions of Echoshield

Achievement	**Description**
Novel Architecture	Dual-branch system combining Structure Analysis (CLAP, ViT, Wav2Vec2) and Artifact Detection (ADM + BiLSTM) branches.
Five Core Innovations	DLSN, AACA, BCBI, MVCL, and Multi-Scale ADM.
Rigorous Protocol	Strict adherence to ASVspoof 2019 LA challenge with strategic data augmentation.
Strong Performance	92.5% accuracy, 6.8% EER after 20 epochs, surpassing baselines.
Ablation Validation	Empirical confirmation of dual-branch superiority over single-branch models.
Efficient Implementation	Memory footprint: 1,678 MB (training), 1,395 MB (inference). Inference: 15.3 ms/sample on A100.

6.1.1 Complete Data Flow Summary

Table 6.2: Tensor Shape Transformations

Stage	**Operation**	**Output Shape**
Preprocessing	Load and Decode	(channels, samples)
	Mono Conversion	(1, samples)
	Resample to 16kHz	(1, new_samples)
	Duration Standardization	(1, 80000)
	Normalization	(1, 80000)
Feature Extraction	STFT	(1, 1025, 157)
	Mel-Scale Conversion	(1, 256, 157)
	Log Compression	(1, 256, 157)
	Delta Features	(1, 256, 157)
	Delta-Delta Features	(1, 256, 157)
	Channel Stacking	(1, 3, 256, 157)
	Temporal Padding	(1, 3, 256, 501)
Branch A (EfficientNet-B7)	Initial Conv	(1, 64, 128, 251)
	MBConv Blocks	(1, 640, 8, 16)
	Final Conv	(1, 2560, 8, 16)
	Global Avg Pool	(1, 2560)
Branch B (Spec-DLSN)	Initial Conv + Pool	(1, 64, 64, 126)
	Residual Blocks	(1, 1024, 8, 16)
	Global Avg Pool	(1, 1024)
AACA	Self-Attention (Branch A)	(1, 2560)
	Self-Attention (Branch B)	(1, 1024)
	Concatenation	(1, 3584)
	Projection Layer	(1, 2048)
BCBI	Reshape for Sequence	(1, 32, 64)
	Bi-GRU Processing	(1, 32, 2048)
	Last Hidden State	(1, 2048)
Classification Head	Multi-layer Perceptron	(1, 1)
	Sigmoid (Inference)	(1, 1) range [0–1]
Projection Head	MLP + L2 Norm	(1, 128)

6.1.2 Performance Summary

The model demonstrated rapid and stable learning, achieving strong results after 20 epochs of training on a Kaggle P100 GPU. The detailed performance metrics are presented in Table 6.3.

Table 6.3: Echoshield Performance Metrics (20 Epochs)

Metric	**Value**
Development (Validation) EER	7.30%
Evaluation (Test) EER	6.80%
Test Accuracy	92.5%
AUC-ROC	0.975
Precision (Bonafide)	93.1%
Recall (Bonafide)	92.2%
Precision (Spoofed)	92.4%
Recall (Spoofed)	93.3%
F1-Score (Macro)	0.924

6.1.3 Architectural Innovations

The five core innovations that distinguish *Echoshield* from existing approaches are detailed in Table 6.4.

Table 6.4: Core Architectural Innovations

Innovation	**Technical Description**
Dynamic Layer Selection Network (DLSN)	Attack-type-aware weighted fusion of CLAP, ViT, and Wav2Vec2 encoder outputs.
Artifact-Aware Cross-Attention (AACA)	Attention mechanism where artifact detection confidence modulates the Structure Branch's focus on relevant features.
Bidirectional Cross-Branch Interaction (BCBI)	Two-way information exchange enabling mutual guidance between Artifact and Structure branches with MI regularization.
Multi-View Contrastive Loss (MVCL)	Composite loss function combining Focal, NT-Xent, Triplet, and Consistency losses for robust multi-perspective training.
Multi-Scale Artifact Detection Module (ADM)	Suite of six parallel forensic extractors (BayarConv, SRM, FFT, Noise CNNs) with BiLSTM temporal modeling.

6.1.4 Comparative Performance

Table 6.5 demonstrates *Echoshield*'s performance relative to established baseline and state-of-the-art models.

Table 6.5: Comparative Performance Analysis

Model	**Parameters**	**Test Accuracy**	**Test EER**	**AUC-ROC**
ResNet50 (Baseline)	25.6M	87.3%	11.8%	0.934
Wav2Vec 2.0 (Fine-tuned)	95.0M	91.2%	7.8%	0.965
Echoshield (20 Epochs)	**361.0M (27.5M trainable)**	**92.5%**	**6.8%**	**0.975**

In conclusion, *Echoshield* successfully handles the difficulty of deepfake audio detection with an innovative, multifaceted technique. The combination of dual-branch processing, advanced fusion mechanisms, and a strong training method has produced a system that performs well while laying the groundwork for future study.

6.2 Limitations

While *Echoshield* has demonstrated strong performance, several limitations must be acknowledged. These are summarized in Table 6.6.

Table 6.6: Current Limitations and Constraints

Limitation	**Description and Impact**
Computational Requirements	361M total parameters require significant GPU resources. Training for 20 epochs on P100 GPU: 12 hours.
Training Duration	Results from only 20 epochs. Training curves indicate potential for further improvement with extended training.
False Positives	Most common in recordings with high background noise or channel distortion (e.g., phone calls).
False Negatives	Occur with high-quality vocoders (HiFi-GAN) and very short utterances (less than 2 seconds).
Dataset Specificity	Core training on ASVspoof 2019 LA. Cross-dataset generalization to ASVspoof 2021 and real-world data is not fully validated.
Edge Deployment	Model size (1,395 MB inference memory) is impractical for mobile/IoT devices without compression.

6.2.1 Error Pattern Analysis

A breakdown of the model's misclassification patterns is provided in Table 6.7.

Table 6.7: Error Pattern Analysis

Error Type	**Primary Causes**	**Estimated Impact**
False Positives (Real as Fake)	High background noise	35-40% of FP
	Low recording quality	25-30% of FP
	Channel distortion	20-25% of FP
False Negatives (Fake as Real)	High-quality vocoders (HiFi-GAN)	40-45% of FN
	Short utterances (less than 2s)	25-30% of FN
	Multi-speaker synthesis	15-20% of FN

These limitations do not diminish the contributions of this work but rather highlight clear directions for future improvement and optimization.

6.3 Future Work

The *Echoshield* project establishes a strong foundation for deepfake audio detection, but several avenues for future research and development remain. These are organized by priority and feasibility in Table 6.8.

Table 6.8: Future Work Roadmap

Priority	**Research Direction**	**Expected Outcome**
High	Extended Training	Train for 50-100 epochs to achieve full convergence and potentially reach 4-5% EER.
High	Hyperparameter Optimization	Systematic search of learning rates, loss weights, and batch sizes for optimal performance.
Medium	Cross-Dataset Validation	Test on ASVspoof 2021, real-world recordings, multilingual datasets to validate generalization.
Medium	Robustness to Degradations	Evaluate under MP3 compression, phone distortion, ambient noise, and reverb.
Medium	Model Compression	Apply knowledge distillation, pruning, and quantization for edge deployment.
Low	Explainability	Develop attention visualization, Grad-CAM, SHAP analysis for interpretability.
Low	Multimodal Integration	Combine with video deepfake detection for comprehensive verification.
Low	Continual Learning	Enable periodic updates with new attack types without catastrophic forgetting.

6.3.1 Detailed Future Directions

1. **Extended Training and Hyperparameter Optimization:** The most immediate next step is to train the model for additional epochs (50-100) to achieve full convergence. Additionally, a systematic hyperparameter search could further optimize performance.

2. **Cross-Dataset Generalization Studies:** Comprehensive testing on out-of-domain datasets (ASVspoof 2021, real-world recordings, multilingual datasets) is essential to validate true generalization capabilities.

3. **Robustness to Audio Degradations:** Systematic evaluation under realistic degradations (MP3 compression, phone channel distortion, ambient noise, reverb) would improve real-world applicability.

4. **Model Compression and Efficiency:** Knowledge distillation, pruning, and quantization techniques should be explored to enable deployment on resource-constrained devices.

5. **Explainability and Interpretability:** Developing visualization methods (attention maps, Grad-CAM, SHAP) would provide insights into decision-making and build trust.

6. **Integration with Multimodal Systems:** Exploring integration with video-based deepfake detection to create comprehensive multimodal verification tools.

7. **Continual Learning and Adaptation:** Investigating strategies for periodic model updates with new attack types without catastrophic forgetting.

8. **Open-Source Release:** Future goal to release model weights, code, and documentation as an open-source project to enable community contributions.

In conclusion, *Echoshield* is a huge step forward in combating audio deepfakes. While problems persist, the new architecture, thorough examination, and encouraging outcomes establish a solid foundation for future developments. The route forward is clear: develop the model, broaden its capabilities, and ensure its effectiveness in protecting against the rising threat of synthetic speech.

References

[1] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *arXiv preprint arXiv:2006.11477*, 2020.

[2] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. *arXiv preprint arXiv:2211.06687*, 2022.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[4] Belhassen Bayar and Matthew C. Stamm. A deep learning approach to universal image manipulation detection using a new convolutional layer. In *Proceedings of the 4th ACM Workshop on Information Hiding and Multimedia Security (IHMMSec '16)*, pages 5–10. ACM, 2016.

[5] Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3):868–882, 2012.

[6] Resemble AI. Resemble ai: Ai voice generator and voice cloning platform. `https://www.resemble.ai`, 2025. Accessed: 2025-01-19.

[7] Play.ht. Voice classifier: Detect ai voices. `https://play.ht/voice-classifier-detect-ai-voices/`. Accessed: Jan. 19, 2025.

[8] Deepware Scanner. Audio analysis result. `https://scanner.deepware.ai/result/963543941cb71b45db52a437502eea4adc7ef686-1709549813/`. Accessed: Jan. 19, 2025.

[9] Hiya. Audio intelligence demo. `https://developer.hiya.com/demos/audiointel/eu/signin`. Accessed: Jan. 19, 2025.

[10] Reality Defender. Ai content moderation and deepfake detection. `https://www.realitydefender.com/`. Accessed: Jan. 19, 2025.

[11] L. Pham, P. Lam, T. Nguyen, H. Nguyen, and A. Schindler. Deepfake audio detection using spectrogram-based feature and ensemble of deep learning models. In *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*, pages 1–5. IEEE, 2024.

[12] Y. Xie, Y. Lu, R. Fu, Z. Wen, Z. Wang, J. Tao, X. Qi, X. Wang, Y. Liu, H. Cheng, L. Ye, and Y. Sun. The codecfake dataset and countermeasures for the universally detection of deepfake audio. *arXiv preprint arXiv:2405.04880*, 2024.

[13] Z. Wang, M. Wu, and T. Yu. Automatic mean opinion score (mos) prediction for speech quality assessment. *arXiv preprint arXiv:2401.13249v2*, 2024.

[14] T. Baoueb, H. Liu, M. Fontaine, J. Le Roux, and G. Richard. Specdiff-gan: A spectrally-shaped noise diffusion gan for speech and music synthesis. *arXiv preprint arXiv:2402.01753*, 2024.

[15] R. Yin, Z. Yang, T. Yao, Z. Peng, X. Li, and Y. Duan. An efficient temporary deepfake location approach based embeddings for partially spoofed audio detection. *arXiv preprint arXiv:2309.03036*, 2023.

[16] L. Pham, P. Lam, D. Tran, H. Tang, T. Nguyen, A. Schindler, F. Skopik, A. Polonsky, and C. Vu. A comprehensive survey with critical analysis for deepfake speech detection. *arXiv preprint arXiv:2409.15180*, 2024.

[17] J. Doe and A. Smith. Is synthetic voice detection research going into the right direction? In *2022 IEEE International Conference on Artificial Intelligence and Signal Processing (AISP)*, pages 1–6, 2022.

[18] E. Togootogtokh and C. Klasen. Antideepfake: Ai for deep fake speech recognition. *arXiv preprint arXiv:2402.10218*, 2024.

[19] N. A. Bhaskaran, M. Srinadh, K. Dhamodhar, and R. Mani Maran. Detecting deep fake voice using machine learning. *Shanlax International Journal of Arts, Science and Humanities*, 11(S3):53–60, July 2024.

[20] S.-Y. Lim, D.-K. Chae, and S.-C. Lee. Detecting deepfake voice using explainable deep learning techniques. *Applied Sciences*, 12(8):3926, 2022.

[21] X. Zhang, J. Chen, L. Wu, and T. Li. Audio deep fake detection with sonic sleuth model. *Computers*, 13(10):256, 2023.

[22] Y. Liu, H. Zhang, Q. Li, and J. Wang. Cross-modality and within-modality regularization for audio-visual deepfake detection. In *Proceedings of the 2024 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–8, 2024.

[23] H. Ouajdi, O. Hadder, M. Tailleur, M. Lagrange, and L. M. Heller. Detection of deepfake environmental audio. In *Proceedings of the 2024 European Signal Processing Conference (EUSIPCO)*, pages 196–200, 2024.

[24] J. Frank and L. Schönherr. Wavefake: A data set to facilitate audio deepfake detection. *arXiv preprint arXiv:2111.02813*, 2021.

[25] A. S. Alshehri, D. Almalki, and S. A. Albaradei. Audio deep fake detection with sonic sleuth model. *Preprints.org*, 2024.

[26] Mohammed Abdeldayem. The fake-or-real (for) dataset. `https://www.kaggle.com/datasets/mohammedabdeldayem/the-fake-or-real-dataset`, 2024.

[27] Audio deepfake detection using deep learning. *Engineering Reports*, 2024.

[28] Mixture of experts fusion for fake audio detection using frozen wav2vec 2.0. *arXiv preprint arXiv:2409.11909*, 2024.

[29] Audio deepfake detection with self-supervised xls-r and sls. In *OpenReview*, 2024.

[30] Towards generalisable and calibrated audio deepfake detection. In *Interspeech 2024*, 2024.

[31] Deepfake audio detection using deep learning. *IEEE Transactions on Audio, Speech, and Language Processing*, 2024.

[32] End-to-end fake audio detection using self-supervised models. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[33] Wav2vec 2.0 for deepfake audio detection: A temporal analysis. *Journal of Machine Learning Research (JMLR)*, 2023.

[34] Temporal feature extraction for deepfake audio detection: A comparative study. *Speech Communication*.

[35] Self-supervised learning for fake audio detection: Combining wav2vec 2.0 and spectral features. *IEEE Access*, 2023.

[36] Deepfake audio detection with temporal convolutional networks and wav2vec 2.0 embeddings. *Journal of Audio Engineering Society*, 2023.